Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Duy Khanh Tran

Master's Thesis
Espoo, October 27, 2026

Supervisor: Prof. Petri Vuorimaa
Major: Computer Science

Draft of 22 October 2026. All seven chapters written. The preamble is a preview; the official Aalto thesis class is applied before submission.

Abstract

A UI transition graph describes the screens of an application and the transitions a user can cause between them. Static extraction of such graphs is well established for Android, where each event handler can be attributed to the widget that registered it and thus to a single window. For web applications, navigation models have been built at design time, extracted statically from server-rendered applications, or obtained by crawling a running application. I found no earlier work that extracts them statically from the route configuration of a client-side framework.

This thesis measures static extraction on six open-source Angular applications with between 14 and 921 declared route entries, using a pinned corpus and four independent evaluation methods. The baseline extractor recovers between 0 and 15 percent of the navigation constructs present in the source as deterministic transitions. For two of the six applications it produces a list of routes with no transitions at all.

Every lost construct is assigned to one defect class. Attribution is the largest class in four of the six applications. In a web application, a navigation site frequently has no single owning screen: it may be located in a shared component, in an injected service, or in a route guard. The thesis formulates attribution as a dataflow analysis over a lattice of screen sets, computed as a fixpoint over component nesting, router outlets and a call graph that takes dependency injection into account. It states a soundness property for the deterministic tier, which it tests mechanically rather than proving, implements the nesting and outlet parts, and evaluates them on the same corpus with the same methods. With attribution, coverage rises in four of the six applications, from 15 to 38 percent on the smallest, and precision on the deterministic tier does not fall. On the larger applications the gain is limited by defects in node extraction, which the thesis also quantifies.

Mutation testing on three of the applications shows that the graph detects 39 of 48 injected source changes, and that eight of the nine changes it fails to detect are removals of a route guard. Attribution does not raise the detection rate for an individual change. It widens the part of the source the graph responds to at all.

Keywords: static analysis; UI transition graph; single-page application; Angular; dataflow analysis; program analysis; software testing

Use of AI tools

Aalto University asks that the use of artificial intelligence tools in a thesis be declared. An AI coding assistant was used in three ways in this work, and I am responsible for the whole of the result.

The extractor under study and the evaluation harness were written with the help of an AI coding assistant. Every number reported in this thesis is produced by running that harness, and the result tables and plots of Chapter 5 are generated from its result files, except the two tables marked as compiled by hand, so that any number can be reproduced from the commits recorded with it. The diagrams in Chapters 1 to 4 were drawn by hand.

The assistant was also used during the literature search, to locate candidate sources and to summarise them. No source is cited on the strength of such a summary. Every entry in the bibliography was checked field by field against the publisher page, Crossref metadata, arXiv metadata or DBLP before it was cited, and sources that could not be checked in this way are kept in a separate block in the bibliography file and are not cited anywhere in the text.

Finally, the assistant was used to draft and edit prose, which I then revised. The research questions, the design of the analysis, the evaluation protocol and the interpretation of the results are my own, as is the responsibility for every claim made here.

Contents

1 Introduction **6**

1.1 Motivation 6

1.2 Research questions and scope 8

1.3 Thesis structure 9

2 Background **10**

2.1 UI transition graphs 10

2.2 Static extraction for Android 11

2.3 Navigation models for web applications 12

2.4 Dynamic state models for single-page applications 13

2.5 Formal foundations 14

2.5.1 Modal transition systems 14

2.5.2 Soundiness 14

2.5.3 Dataflow analysis and abstract interpretation 14

2.6 Static analysis of JavaScript and TypeScript source 15

2.6.1 Analyses over a source-level syntax tree 15

2.6.2 Abstract interpretation of JavaScript 16

2.6.3 What TypeScript's types contribute 16

2.6.4 Frameworks, not the language, are the obstacle 16

2.7 Angular routing, dependency injection and reactive state 17

2.7.1 Routes 18

2.7.2 Components and templates 18

2.7.3 Navigation sites 19

2.7.4 Dependency injection 19

2.7.5 Reactive state 21

2.8 Reactive programming and the limits of static dataflow 21

2.8.1 Propagation order and glitch freedom 21

2.8.2 Reactive code and the tools developers have 22

2.9 Mutation testing 22

2.9.1 The coupling effect . . . 23
2.9.2 Mutation for JavaScript and web applications . . . 23
2.9.3 Reporting a mutation experiment . . . 24
2.10 The gap . . . 24

3 Static navigation analysis and scope attribution 26
3.1 Graph model and soundness . . . 26
3.2 Scope attribution . . . 28
3.2.1 The defect being addressed . . . 28
3.2.2 Scope . . . 28
3.2.3 Input relations . . . 31
3.2.4 Guards as program points . . . 31
3.2.5 Boundary . . . 32
3.2.6 Anchored edges . . . 33
3.3 Implementation . . . 33

4 Evaluation setup and research material 35
4.1 Corpus selection and validity . . . 35
4.1.1 Selection . . . 35
4.1.2 Validity of an open-source corpus . . . 36
4.2 Metrics . . . 37
4.3 Evaluation methods . . . 38
4.3.1 Preconditions . . . 39
4.3.2 Corrections made to the protocol . . . 39
4.3.3 Reproducibility . . . 41

5 Results 42
5.1 Baseline extraction (RQ1) . . . 42
5.2 Defect classes (RQ2) . . . 42
5.3 After attribution (RQ3) . . . 44
5.4 Change sensitivity (RQ4) . . . 48

6 Discussion 52
6.1 Key findings . . . 52
6.2 Limitations and threats to validity . . . 55
6.3 Declared approximations . . . 57
6.4 Transfer to other frameworks . . . 59
6.5 Future work . . . 61

7 Conclusion **64**

Chapter 1

Introduction

A user interface transition graph is a model of an application in which the nodes are the screens a user can see and the edges are the transitions a user action can cause between them. For each edge, the graph records the event that triggers it, the condition under which the transition is available, and the screen it leads to. A graph of this kind makes it possible to answer questions that are difficult to answer from the code alone, for example which screens are reachable from a given screen, or which parts of the user interface are affected when a particular component changes.

Transition graphs have been used in automated testing for a long time. In the Android domain they are used for test generation and for guiding exploration [77, 39, 14], and in web testing they form the basis of crawling-based test derivation [50, 51]. They are also useful for change impact analysis [4, 12], because each transition can be traced back to the code that implements it, and from there to the tests that cover it.

1.1 Motivation

There are two general ways to obtain a transition graph. A dynamic approach executes the application and records the states and transitions it observes. A static approach reads the source code and derives the graph without running anything. Their limitations differ. A dynamically built graph contains only the states the crawler managed to reach, which depends on the available test data, authentication, and the exploration budget, and the graph has to be rebuilt for every new version. A statically built graph covers everything the source declares and can be recomputed on each commit without a running system, but it is limited to what can be decided from the source, and it must over-approximate wherever a target cannot be resolved.

For Android, static extraction is a well-established technique. The window transition graph work derives windows and the transitions between them from bytecode, and resolves the control flow of callbacks to the widget that registered each handler [77, 76, 75]. For the web, navigation models have been extracted statically from server-rendered frameworks, in which pages and links are declared in templates and configuration files [61, 9, 67]. Modern single-page applications, however, have almost always been modelled by executing them. Crawljax builds its state flow graph by firing events against the DOM of a running application [50], and later tools in the

same line improve the exploration strategy without changing where the model comes from [51, 72]. A survey of web testing published in 2025 collects the model-based tools of the past decade [36]. Among the tools it lists, no model is derived from source code rather than from execution. During the literature search for this thesis, only one static analysis of a modern client-side framework was found; it builds a component dependency graph for React and does not address routing [38].

When the dynamic approach was developed, a single-page application had no declarative route table, so the DOM was the only thing to observe (Section 2.4). Client-side routing frameworks changed this. Angular, React Router and Vue Router all declare routes as data in the source code, connect navigation to those routes through template attributes and service calls, and attach guards to routes as named functions [26]. The artefact needed for static analysis now exists, but how well static extraction works on real applications built with these frameworks, and where it fails, has not been measured.

A statically extracted graph also has consumers outside model-based testing. Tools that drive a graphical user interface with a large language model explore an application without any model of its navigation structure. GPTDroid treats mobile GUI testing as a question-and-answer loop, passing the current screen to the model and replaying the action it returns, and reports higher activity coverage and more bugs than the best baseline on 93 applications [46]. AutoDroid gives the model application-specific knowledge in addition to its commonsense knowledge, and obtains that knowledge by automated dynamic exploration [71]. LLMDroid uses the model to guide exploration towards coverage the test run has not reached [68]. The approach has been applied to web interfaces with a vision-language model and curiosity-driven exploration [70], and a survey of large language models in software testing covers the wider area [69]. Coding agents lack the same kind of structure one level up, where the context is a repository rather than a screen: the SWE-bench issues typically require coordinated edits across functions, classes and files [31], agent performance depends strongly on the interface through which the repository is navigated [74], and retrieving code from elsewhere in the repository improves completion over an in-file baseline [79]. A graph that states which screens an application has and which transitions connect them is one source of such structure, and it can be produced without running the application.

This thesis carries out that measurement for Angular and proposes an analysis for the largest class of failures it finds. What is proposed is a method rather than a product: scope attribution is stated as a dataflow analysis over a lattice of screen sets, and nothing in its formulation depends on a particular extractor.

A method of this kind cannot be assessed without being instantiated, because the questions asked here are empirical: how much of the navigation in real applications it recovers, where it fails, and whether the resulting graph responds to a change in the source. The instrument used is *uigraph*, an open-source static analyser for Angular that I developed before this work began. It plays the part the subject of study plays in an experiment, not the part of a result: the figures reported here describe what the analysis recovers when realised this way, and the limits they expose are limits of the method and of this instantiation, which Chapter 6 separates.

1.2 Research questions and scope

The thesis addresses four research questions. The exact measures are defined in Section 4.2.

RQ1. *How much of the navigation in Angular applications can static analysis recover as deterministic transitions?*

This is measured as the fraction of navigation constructs in the source code that are witnessed by at least one deterministic (*must*) transition in the extracted graph.

RQ2. *Where does static extraction fail, and why?*

The observed failures are grouped into defect classes. A class is accepted only if at least two independent evaluation methods confirm it. Each construct that is not recovered is then assigned to one class, so the classes can be compared by size within each application.

RQ3. *How does scope attribution affect the coverage and precision of static navigation analysis?*

Scope attribution is the analysis proposed in this thesis. For a navigation site that does not belong to any single screen, it computes the set of screens from which the site can be triggered. One commit of the instrument is run twice on the same inputs, with the attribution pass disabled and enabled, so that nothing but the pass differs between the two runs. Coverage is the change in recovered transitions. Precision is checked on the deterministic tier, which must not decrease.

RQ4. *Is the extracted graph sensitive enough to source changes to support change impact analysis?*

This is measured with mutation testing. Known changes are inserted into the source, and the graph is checked for the expected difference.

RQ1 and RQ2 are answered by measuring the extractor on a corpus of six open-source Angular applications. The applications are pinned to specific commits and range from 14 to 921 declared route entries. Four evaluation methods are used, chosen so that they do not share the same blind spot. RQ3 is addressed by formulating attribution as a dataflow analysis over a lattice of screen sets, implementing it, and evaluating the result on the same corpus with the same methods. RQ4 is answered on the graph produced after attribution has been applied.

The thesis has three limits of scope. It does not claim completeness. When a navigation target is computed at run time, the full set of transitions cannot be determined statically in general. The thesis therefore states a conditional soundness property over a declared scope, tests it mechanically against the source at every deterministic edge it produces, and measures recall empirically. The property is not proved, and Chapter 5 reports the one edge in the corpus that violates it. This follows the position of Livshits et al., who argue that a practical static analysis should be described as sound modulo an explicit list of the language features it approximates [47]. It also does not measure the effect of the graph on coding agents or on any other consumer. Such a measurement would require a controlled experiment and is

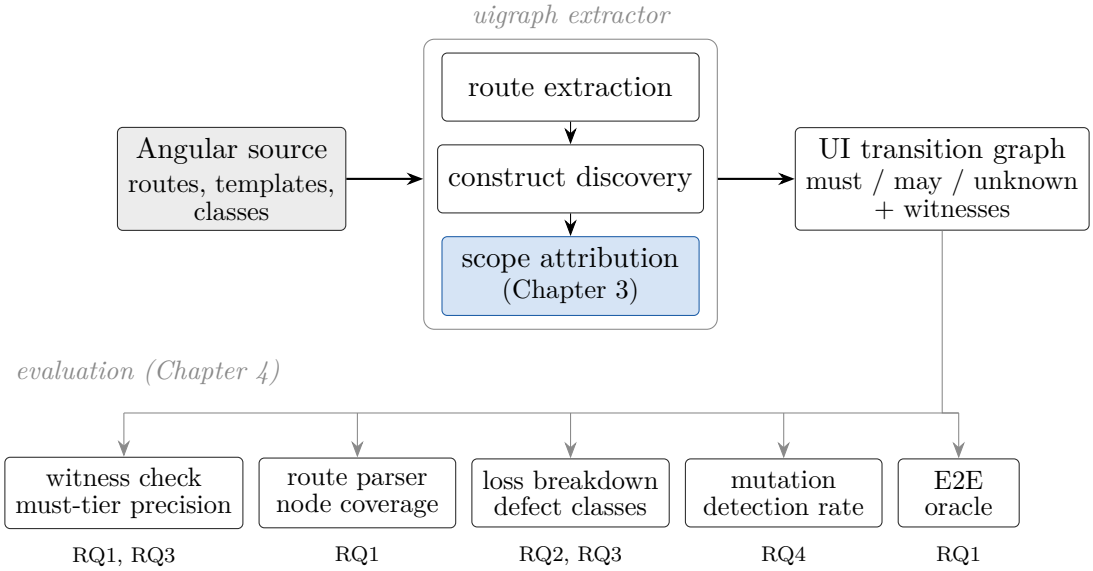


Figure 1.1: Overview of the study. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. Five measurements, each tied to the research questions it answers, evaluate the graph.

left for future work. Finally, the results apply to Angular only. The graph model does not depend on the framework, and the tool has adapters for other frameworks, but all measurements reported here concern Angular applications.

1.3 Thesis structure

Figure 1.1 shows how the parts of the study fit together.

Chapter 2 presents the background. It introduces the terminology used for transition graphs in different research communities, reviews static extraction for Android, navigation models for web applications and dynamic state models for single-page applications, and describes the Angular mechanisms the analysis has to handle together with the program analysis ideas it builds on. Chapter 3 presents the contribution: the graph model and its soundness claim, the scope attribution analysis, and its implementation. Chapter 4 describes the evaluation setup: the corpus and why it can be considered valid, the metrics, and the four evaluation methods. Chapter 5 presents the results for each research question. Chapter 6 discusses the results, the limitations of the study, the constructs the analysis approximates, how the approach transfers to other frameworks, and future work. Chapter 7 concludes the thesis.

Chapter 2

Background

This chapter provides the background needed for the rest of the thesis. Section 2.1 fixes the terminology, since the same kind of graph is called by different names in different communities. Sections 2.2 to 2.4 review the three lines of prior work that are closest to this thesis: static extraction for Android, navigation models for web applications, and dynamic state models for single-page applications. Section 2.5 summarises the three formal ideas the analysis builds on. Section 2.6 reviews static analysis carried out over a source-level syntax tree, and what the type annotations of a TypeScript program do and do not contribute to it; this is the tooling style used by the analysis in Chapter 3. Section 2.7 introduces the Angular mechanisms that the analysis has to deal with and the program analysis problem each of them raises, with dependency injection treated at length because it is the mechanism that hides the most control flow. Section 2.8 reviews reactive programming and the boundary it puts on a static analysis. Section 2.9 reviews mutation testing, which is the method used in Chapter 4 to answer RQ4. Section 2.10 states the gap that the thesis addresses.

2.1 UI transition graphs

A UI transition graph is a directed graph in which the nodes represent user-visible states of an application and the edges represent transitions that a user action can cause between those states. Figure 2.1 shows a small example in the model used in this thesis.

Several research communities have studied this kind of graph, and each uses its own terminology. In Android static analysis it is called a *window transition graph* [77], or, at a finer level of detail, an *activity transition graph* [14] or a *screen transition graph* [39]. In the Ajax crawling literature the term is *state flow graph*, where a state is an abstraction of the DOM [50]. In web engineering the usual terms are *navigation model* and *navigation state machine* [61, 18]. Recent work on software agents uses the term *UI transition graph*. This thesis uses the last of these and treats the others as equivalent for its purposes, while noting that they differ in what is counted as a state and in how the graph is constructed.

More important than the naming is how the graph is obtained: *statically*, from source code or binaries without executing the application, or *dynamically*, by running

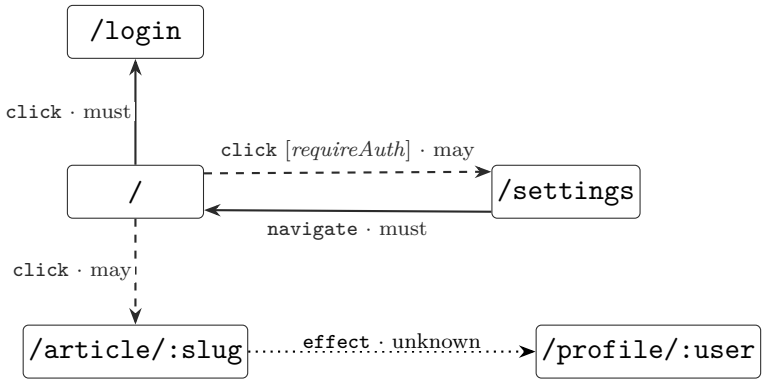


Figure 2.1: A small UI transition graph in the model of this thesis. Each edge carries an event, an optional guard in brackets, and a modality. A *must* edge always fires when its code runs, a *may* edge is conditional, here on a route guard or on a target that is only partly known, and an *unknown* edge is one whose firing condition cannot be determined statically, such as a navigation inside a signal effect.

and observing it. It also matters whether the graph represents *guards*, that is, whether each edge records the condition under which the transition is available or only that the transition exists. Table 2.1 classifies the work reviewed in the following sections according to both distinctions.

2.2 Static extraction for Android

The largest body of static work on transition graphs concerns Android. Yang et al. introduced the window transition graph, in which the nodes are windows (activities, dialogs and menus) and the edges are transitions caused by callbacks that run on the UI thread [77]. A later version of the same work adds an explicit window stack so that the behaviour of the back button can be modelled [75]. A related analysis by the same group constructs a callback control-flow graph whose nodes are pairs of a handler and the widget it belongs to [76]. StoryDroid builds an activity transition graph from `startActivity` calls and uses a simple lexical rule to attribute code inside an inner class to the enclosing activity [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration and evaluates the result against manually produced ground truth for 36 applications [39]. UI-CTX identifies the problem of finding the correct code context for a widget's behaviour as its first challenge and solves it with a backward data-flow analysis from the handler to the place where the widget is created [44].

These approaches have in common the way a navigation is attributed to a screen. In Android, an event handler is registered on a widget object, and at run time that object belongs to exactly one window. Attribution can therefore be treated as an object resolution problem: the widget identifier is resolved, usually by constant propagation of the `getId()` call, and the window is then known [76]. The problem is well defined because the platform's object model guarantees a single owner.

The limits of this approach become visible in ProMal. Its authors report that a

Table 2.1: Prior work closest to this thesis, classified by target, extraction mode, guard representation, and the ground truth used for evaluation.

Work	Target	Static	Guards	Ground truth
Yang et al. [77, 75]	Android windows	yes	callbacks	manual, 6 apps
Yang et al. [76]	Android callbacks	yes	context	manual vs. dynamic
StoryDroid [14]	Android activities	yes	no	hand-built, 10 apps
GoalExplorer [39]	Android screens	hybrid	intents	manual, 36 apps
ProMal [45]	Android windows	hybrid	no	40 apps, F1
UI-CTX [44]	Android widgets	yes	branches	2000 labelled widgets
Ricca and Tonella [61]	server-rendered pages	semi	yes	manual
WebUml [9]	server-rendered pages	hybrid	yes	case study
Tkachuk and Rajan [67]	Struts/JSP	yes	outcome	not stated
Crawljax [50]	Ajax DOM states	no	no	manual
FeedEx, FRAGGEN [51, 72]	DOM states	no	no	coverage
Kurant [38]	React components	yes	n/a	F1, no routing

purely static window transition graph loses precision on user-interface code that is shared between contexts, such as a dialog that several activities reuse. They address this by combining the static analysis with runtime traces and a learned model [45]. This is the case in which no single owning window exists, and here the Android work moves away from static analysis.

2.3 Navigation models for web applications

Navigation models for the web were first extracted from server-rendered sites, well before single-page applications became common. Ricca and Tonella modelled a web application as a set of pages, frames and forms connected by links and form submissions. Their model includes conditional edges that carry a boolean condition on input values, and it was derived semi-automatically using a crawler together with annotations by an analyst [61]. WebUml recovers a state diagram from HTML, script and server-side pages by combining static source analysis with mutation-driven execution, and derives guards by control-flow analysis of the server code [9]. A patent assigned to Fujitsu describes a fully static method for extracting a navigation

model from Struts and JSP sources, including pages that are referenced but do not exist [67]. On the verification side, Haydar et al. model a web application as a system of communicating automata derived from recorded browsing sessions [28], and SiteHopper builds abstracted navigation state machines on the fly for model checking [18]. In addition, design-time notations such as OOHDM, WebML and UWE describe the navigation structure of a site before it is implemented rather than extracting it from code.

This work shows that navigation models, including guards, have been extracted statically from web applications. However, all of these models concern server-rendered frameworks, where a page corresponds to a file and a link to an anchor element. None of this work deals with a client-side router, a component tree, or dependency injection.

2.4 Dynamic state models for single-page applications

When web applications moved to Ajax and later to client-side frameworks, work on model construction moved to run time. Crawljax executes user events against a running application, compares the DOM before and after each event, and builds a state flow graph whose nodes are equivalence classes of DOM states [50]. Later work in this line changed the exploration strategy but not the source of the model. FeedEx uses feedback from the graph built so far to direct further exploration [51], and FRAGGEN abstracts states as hierarchies of DOM fragments in order to generate better tests [72]. The survey by Kertusha et al. covers the model-based web testing tools of the past decade [36]. Of the tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application.

The explanation for this seems to be historical. In an Ajax application there was no declarative route table, and one URL could represent many states, because navigation was implemented by handlers that modified the DOM. The DOM was therefore the only observable that distinguished one state from another. Static analysis of the JavaScript of that period, with its dynamic dispatch and handlers constructed at run time, was in practice not feasible. So the dynamic approach was used because there was no static artefact to analyse.

Client-side routing frameworks changed this situation. The route table is now a declared data structure in the source, a navigation is an attribute or a method call that names a route, and a guard is a named function attached to a route. Only one static analysis of such a framework was found during the literature search for this thesis, namely Kurant's component dependency graph for React, and it stops at the component rendering graph without addressing routing [38]. The work in this thesis is located between that analysis and the dynamic tools described above.

2.5 Formal foundations

The analysis builds on three ideas from the literature: a transition system in which edges carry a modality, a stated position on what soundness can mean for a practical analysis, and dataflow analysis as the computational framework.

2.5.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required (*must*) or merely permitted (*may*) [42]. The graph model in Chapter 3 adopts this distinction and adds a third label, *unknown*, for edges whose modality could not be determined. The distinction has practical consequences: the thesis makes a soundness claim about the must tier only, and the evaluation treats the tiers differently, since the evidence behind a may edge establishes less than the evidence behind a must edge.

Carrying a modality on the edge also changes what an unresolved construct does to the graph. An analysis with a single edge kind has two options when it cannot decide whether a transition occurs: emit the edge and lose precision, or drop it and lose recall. With a modality the same situation produces an edge whose label records how much was established, and a consumer can select the tier it needs. The tiers are reported separately in Chapter 5 for this reason.

2.5.2 Soundiness

Livshits et al. argue that practical static analyses are sound only with respect to a subset of the language, that this is unavoidable in practice, and that the appropriate response is to state explicitly which language features are approximated rather than to claim soundness in general [47]. This thesis follows that recommendation: it states its soundness property only over a declared scope, lists the approximated Angular constructs with their measured frequency in the corpus, and measures recall empirically. The literature reviewed in Sections 2.6 to 2.8 gives the specific reasons why a general claim is not available for this setting. Type annotations in TypeScript are deliberately not guarantees, framework configuration decides which implementation a call reaches, and the firing condition of a reactive computation is not written in the program text.

2.5.3 Dataflow analysis and abstract interpretation

Kildall showed that many program analyses can be stated as the computation of a fixpoint over a lattice of abstract values, where information is combined with a join operation and propagated along the edges of a graph until nothing changes [37]. If the lattice has finite height and the transfer functions are monotone, the computation terminates and its result does not depend on the order in which nodes are visited. Cousot and Cousot gave the general theory behind this approach as

abstract interpretation [16], and Nielson et al. give a textbook account [54]. Scope attribution in Chapter 3 is an instance of this framework. The abstract values are sets of screens, the graph is the call graph extended with component nesting, and the join is the union of scopes.

The framework is independent of the language being analysed, and it has been applied to JavaScript as well; Section 2.6 describes that work. What a particular instance needs is a lattice whose height is bounded and a propagation graph. Both are available here: the set of screens is finite, so the lattice of screen sets has a height equal to the number of screens, and the call graph with component nesting supplies the propagation graph. Where the propagation graph itself is incomplete, because a call could not be resolved, the result is incomplete in a way the framework does not detect, which is why the scope has to be declared.

2.6 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 reads Angular source with the TypeScript compiler API, so it works on a source-level abstract syntax tree rather than on a compiled artefact. This section reviews that tooling style, the abstract interpretation work that put lattice-based analysis of JavaScript on a formal footing, what the type annotations of a TypeScript program contribute, and the evidence that frameworks rather than the language are what limits such analyses in practice.

2.6.1 Analyses over a source-level syntax tree

Analysing the syntax tree of the source, rather than a bytecode or an intermediate representation, is a long-established style. Baxter et al. detect exact and near-miss code clones by comparing abstract syntax trees, using a variant of the common-subexpression detection used in compilers, and apply the technique to a C system of about 400 000 lines [8]. The approach keeps the analysis close to what the developer wrote, which matters when the result has to be reported back at a source location.

SPOON makes this style into a reusable library. It exposes a metamodel of Java source and lets an analysis or a transformation be written in plain Java, without modifying a compiler and without learning a separate formalism [57]. The TypeScript compiler API occupies the same position for TypeScript: it gives a typed syntax tree, a symbol table and a type checker over an ordinary program, which is what allows the Angular adapter to resolve a constant reference in a route table or the declared type of an injected field. The cost of the style is that the analysis sees the program as written, including the parts that a compiler would later rewrite or erase.

2.6.2 Abstract interpretation of JavaScript

Lattice-based analysis of JavaScript itself has a formal treatment. Jensen et al. present a static analysis based on abstract interpretation that infers detailed and sound type information for JavaScript, covering the full language including its object model and the built-in functions, and propose it as a foundation for developer tools [29]. The work shows that the framework of Section 2.5 applies to JavaScript at the level of individual values, which is a considerably harder problem than the one addressed in this thesis. Scope attribution uses the same machinery over a much coarser domain. Its elements are sets of screens, not abstractions of heap objects, so the height of the lattice is the number of screens and does not depend on the structure of the heap.

2.6.3 What TypeScript's types contribute

Angular applications are written in TypeScript, so an analysis can read declared types instead of inferring them. The type system is deliberately unsound. Bierman et al. give a precise definition of it [10], and Richards et al. describe the annotations as unsound by design [62]. That deliberate unsoundness means a declared type is evidence about the program rather than a guarantee about its execution.

The empirical picture is consistent with that. Gao et al. took bugs that had been fixed in the histories of JavaScript projects, added type annotations to the buggy versions, and checked whether a type checker would have reported them; both Flow and TypeScript detected about 15% of these bugs [25]. Bogner and Merkel mined 604 GitHub projects, 299 in JavaScript and 305 in TypeScript, together more than 16 million lines of code, and found better code quality and understandability for the TypeScript projects, but no lower bug proneness and no shorter bug resolution time; heavier use of the `any` type correlated weakly with worse measurements [11]. The declarations that describe untyped libraries can themselves be wrong: TSCheck compares a declaration file against the library it describes by capturing the runtime object state after initialisation and adding a light static analysis of function code, and found 142 errors in the declarations of 10 libraries [22].

What follows for an analysis of Angular code is a division of labour. A declared type narrows the set of candidate implementations of a call, which is useful and is what the call graph construction in Section 2.7 relies on. It does not establish that the call reaches exactly one of them, so an unresolved or ambiguous type has to widen the result rather than be trusted. In the graph model this appears as the difference between the must tier and the may tier.

2.6.4 Frameworks, not the language, are the obstacle

The practical obstacle to analysing a client-side application is the framework around the code, not the language. Madsen et al. observe that JavaScript applications depend on large frameworks and libraries whose behaviour a static analysis has to know, that hand-written models of them are tedious to produce, incomplete and

sometimes wrong, and that the alternative is to infer library behaviour from the way the application uses it; they combine pointer analysis with such a *use analysis* and evaluate it on 25 Windows 8 JavaScript applications [48]. A later analysis by Madsen et al. addresses implicit control flow directly. Their *event-based call graph* for Node.js represents listeners that run when an event fires, and the analyses that build it find event-handling defects such as events that no listener handles and listeners registered too late [49]. An Angular application has the same shape of problem, with the router, template event bindings and observable subscriptions in place of the Node.js event emitter.

The closest peer-reviewed static analysis of a framework in the Angular family is Aurebesh, by Ocariza et al. [55]. It defines a consistency model for web applications built on an MVC framework and detects identifier and type inconsistencies between models, views and controllers, reporting a recall of 96.1% and a precision of 100%, and finding 15 real bugs. The work concerns AngularJS 1.x, which is a different framework from the Angular versions this thesis analyses, with a different template language, a different component model and a different injector. What the two have in common is the problem of relating an identifier in a template to the code that defines it, which Aurebesh solves for a consistency check. Aurebesh targets consistency between templates and controllers rather than navigation, and reports no transition graph.

2.7 Angular routing, dependency injection and reactive state

The analysis presented in Chapter 3 operates on Angular applications, and most of its difficulties arise from particular Angular mechanisms. This section introduces those mechanisms and relates each of them to the program analysis problem it creates. The Angular documentation is the reference for the framework details [26]. Listings 2.1 to 2.3 show the constructs discussed below in a small example, which later chapters return to.

Listing 2.1: A route table with the constructs the analysis must handle: a guarded route, a parent route whose children render in its outlet, a lazily loaded route table and a path held in a constant.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root component's template. The header sits next to the outlet, so its links are present on every screen but belong to none of them.

```
<app-header />                  <!-- shell: rendered on every screen -->
<router-outlet />               <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Listing 2.3: Navigation from an injected service. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

2.7.1 Routes

An Angular application declares its screens as a tree of route objects. Each route has a `path` pattern and, in addition, either a `component` to render, a `redirectTo` target, or a list of `children`. A route that has children but no component acts as a path prefix and does not itself correspond to a screen. Routes can be loaded lazily using `loadChildren` or `loadComponent`; in that case the child routes are defined in another module or file that is resolved when the application is built. Routes may also have guards, namely `canActivate`, `canMatch` and `canDeactivate`, which are functions or injectable classes that return either a boolean or a `UrlTree` representing a redirect. Route paths are strings, but they are not necessarily string literals. Applications that keep their route table in one place often write `path: routes.about.path`, and resolving such a path requires following the constant to its definition.

2.7.2 Components and templates

Each screen is rendered by a component, which is a class together with a template. A template can instantiate other components by their selectors, so the components of an application form a nesting graph. A template can also contain a `<router-outlet>` element, which marks the position where the router inserts the component of the currently active child route. Components that are above the outlet in the nesting graph make up the application shell and are present on every screen, whereas components below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make the nesting conditional. Templates can additionally be passed around as values of type `TemplateRef` and rendered in a different component, and components can be created programmatically at run time.

2.7.3 Navigation sites

In a template, navigation is expressed with the `routerLink` directive, whose value is either a literal path or an array of path segments. In class code, navigation is performed by calling `Router.navigate` or `Router.navigateByUrl`; the target may be given relative to the current route through the `relativeTo` option. Both kinds of navigation site can appear in any component, in any injectable service, and in guards, interceptors and resolvers. The source code does not state anywhere which screens a given site can be triggered from; this information has to be derived.

2.7.4 Dependency injection

Angular creates services and passes them to the components and services that declare them as dependencies. This is dependency injection, a form of inversion of control: a class does not construct or look up its collaborators, but receives them from a container [32, 24]. A dependency is requested by type or by an injection token, and the same token can be bound to different implementations in different injector scopes, using `useClass`, `useExisting` or a factory function.

The pattern is not specific to Angular and is widely used. Yang, Tempero and Melton developed operational definitions of dependency injection and techniques for detecting its use from source code, and applied them to 34 open-source Java applications [73]. Their motivation is the usual argument for the pattern, that it improves extensibility, modifiability, testability and reusability. The configuration that dependency injection introduces is itself an object of static analysis: Laigner et al. catalogue 12 dependency injection anti-patterns together with refactorings, and detect them with a static analyser at a relative recall of 92.19%, reporting that at least 9 of the anti-patterns occur frequently in open-source and industrial projects [40]. Dependency injection configuration is therefore readable statically; it does not state which binding a given call site will see.

For a static analysis this turns a simple method call into a dispatch problem. A call such as `this.nav.goHome()` can only be linked to the method it invokes after the type of `nav` and the binding of that type in the relevant injector have been resolved. This is the call graph construction problem known from object-oriented languages, where a virtual call must be resolved to its possible targets. Class hierarchy analysis resolves such a call using the declared type and its subclasses [17]. Rapid type analysis narrows the result to classes that are actually instantiated [5], and points-to analysis narrows it further by tracking which objects each variable may hold [2]. Grove and Chambers place these algorithms in a common framework that trades precision against cost [27]. For JavaScript, Feldthaus et al. show that an approximate call graph, which tracks functions only by the property names they are stored under, is precise enough for practical tools [23].

Dependency injection adds a further difficulty, because the binding of a type to an implementation is not written in ordinary code but in framework configuration. The Java literature has measured what this costs a general analysis and what recovers it, and Table 2.2 lists the analyses that model framework behaviour explicitly. Sridharan

Table 2.2: Static analyses that model framework behaviour explicitly, and the mechanism each of them resolves.

Work	Framework	Mechanism modelled
F4F [65]	Java web frameworks	reflective calls, from a specification generated from code and configuration
JackEE [3]	Java enterprise frameworks	components and entry points, declaratively
Jasmine [13]	Spring	dependency injection and aspect-oriented programming
Aurebesh [55]	AngularJS 1.x	identifiers shared between template and controller

et al. report that the heavy use of reflection in frameworks makes static taint analysis ineffective, and that adding support for one framework at a time does not scale; their tool instead generates a specification of framework behaviour from the application code and its configuration files, which an unmodified taint engine then consumes, and this found 525 new issues across nine benchmarks [65]. Antoniadis et al. report that major Java analysis frameworks offer almost no support for enterprise applications, capture the concepts those frameworks share, such as components and entry points, in a declarative and largely framework-independent form, and measure about four times higher completeness and about a sixfold speedup against the most precise conventional analysis [3]. Landman et al. put a bound on what can be hoped for: in a literature review and corpus study of Java reflection they find that reflection cannot be ignored in 78% of projects and that resolving it requires unsound assumptions [41].

The closest analogue to resolving Angular's injectors is Jasmine, by Chen et al. [13]. Spring's core mechanisms, dependency injection and aspect-oriented programming, rely on dynamic language features, and the consequence is that a conventional call graph of a Spring application is missing edges. Jasmine extends Soot so that the call graph of a Spring program becomes more complete, and was evaluated on micro-benchmarks and on 18 real Spring programs; it improves on tools built with Soot and with Doop, and in combination with FlowDroid it exposed 12 sensitive-data leak paths. The structural problem is the one that occurs in Angular: an injected field has a declared type, the binding of that type lives in framework configuration, and an analysis that does not read the configuration cannot connect the call to a body. The difference is in where the configuration lives and how far its effect reaches. Angular binds tokens in provider arrays that belong to a module, a route or a component, so the same token can resolve to different implementations depending on where the injection happens. Resolution is relative to an injector rather than global, which makes the provider hierarchy part of the input to the analysis.

The analysis in Chapter 3 follows the advice this literature converges on, which is to model the framework explicitly rather than to treat its configuration as ordinary

code. It reads Angular's provider declarations and resolves an injected type in the style of class hierarchy analysis. Where a token can be bound to more than one implementation, all of them are considered.

2.7.5 Reactive state

Recent versions of Angular include signals and effects. A signal is a value that changes over time, and an effect is a computation that runs again whenever a signal it read changes. This is an instance of reactive programming, which goes back to functional reactive programming [21] and has been surveyed by Bainomugisha et al. [6]. In reactive systems the dependencies between values are recorded at run time, and the runtime propagates updates without explicit calls in the program text. So when a navigation call is placed inside an effect, the condition under which it fires is a data dependence in a reactive graph and not a control-flow condition. A static analysis can find which signals the effect reads, but not when the effect runs. Angular applications also use RxJS observables for the same purpose, in router event subscriptions, HTTP responses and state containers. Section 2.8 reviews what the reactive programming literature establishes about this boundary.

2.8 Reactive programming and the limits of static dataflow

The reactive style gives the runtime, rather than the program text, control over when a computation runs. This section reviews the propagation discipline that reactive runtimes implement, and the evidence about how well reactive code can be reasoned about with tools.

2.8.1 Propagation order and glitch freedom

A reactive program is a graph of values in which an update to one value has to be propagated to the values that depend on it. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language, so that an expression evaluates to a node in a dataflow graph [15]. Their account names the central hazard: a *glitch* occurs when a signal is recomputed before all of the signals it depends on have been brought up to date, which makes an intermediate value observable that no consistent state of the program ever had. Their runtime prevents glitches by assigning each signal a height and updating signals in order of increasing height, which is a topological order and therefore requires the dependency graph to be acyclic. Drechsler et al. carry the property into a distributed setting with a change-propagation algorithm that needs no central knowledge of the dependency topology, avoids propagating updates that cannot change a result, and preserves glitch freedom [20]. An Angular signal graph, as described in Section 2.7.5, records its dependencies as they are read, so it is the kind of structure that these propagation algorithms are defined over.

The consequence for a static analysis is that the order in which reactive computations run is a property of the graph that the runtime builds, and the graph is built from the reads that actually happen. The set of signals a given effect may read can be over-approximated from the source. The number of times the effect runs, and the state of the application when it runs, cannot be, because that depends on which writes occur and in what order.

2.8.2 Reactive code and the tools developers have

The empirical literature on reactive programming reports a split between reading code and reasoning about its behaviour. Salvaneschi et al. ran an experiment with 127 subjects comparing reactive programming against object-oriented code using the Observer pattern, and found comprehension significantly better in the reactive version [64]. The same group reports that the tool support is the weak side. Reactive programs, they argue, need debugging concepts of their own, because a conventional debugger shows a call stack while the question is about a dataflow graph; their Reactive Inspector implements such concepts and outperformed conventional debugging in a controlled experiment [63].

The same picture holds for the Rx family of libraries, which Angular applications use for asynchronous data. Banken et al. report that conventional debuggers have no support for Rx and that developers fall back on inserting console logging; their RxFiddle visualises the structure and the values of an Rx dataflow, and in an experiment with 111 developers it let them finish debugging tasks faster [7]. Alabor and Stolze interviewed 10 professional engineers and observed 4 of them debugging RxJS code, and found that they rely on manually added trace logging rather than on specialised tools; the study is a workshop paper with a small sample, so it indicates rather than establishes the practice [1]. The surface that a tool would have to cover is large: Zimmerle et al. mined RxJava, RxJS and RxSwift usage on GitHub and Stack Overflow and found that 95.2% of the Rx operators are used in practice, with the stream abstraction itself the largest of the 23 Stack Overflow topics they identified, at 36.4% of the posts [80].

An analysis that aims to recover navigation from an Angular application inherits this boundary. A navigation placed in an effect or in a subscription has a firing condition that is a data dependence in a reactive graph. The read set is visible in the source and the firing is not. Chapter 3 therefore specifies the modality *unknown* for such edges and keeps the read set as the symbolic guard, which is the treatment the soundiness position of Section 2.5 prescribes for a construct that cannot be decided. No pass in this version emits that modality.

2.9 Mutation testing

RQ4 asks whether the extracted graph is sensitive enough to changes in the source to be useful for change impact analysis. The method used to answer it is mutation testing, applied in an unusual direction: the mutants are inserted into the application

source, and the artefact whose reaction is measured is the extracted graph rather than a test suite. This section reviews the technique, the evidence for the assumption it rests on, its use on JavaScript and web applications, and the reporting practice that Chapter 4 follows.

2.9.1 The coupling effect

Mutation testing inserts small syntactic changes into a program and measures how many of them a test suite detects. The original argument for it is the *coupling effect* stated by DeMillo et al., namely that tests which uncover simple errors are often also effective at uncovering much more complex errors [19]. Without that assumption a mutant would say nothing about the faults a developer actually writes.

Jia and Harman give the standard survey of the field, covering approaches, tools and empirical results; their trend analysis suggests that the techniques and the tools are reaching maturity and applicability [30]. Papadakis et al. survey the following decade of work on the fundamental problems of the technique and list the problems that remain open [56]. The evidence that mutants are a usable proxy comes from Just et al., who used 357 real faults in 5 open-source applications totalling 321 000 lines of code and found that mutant detection correlates significantly with real-fault detection, independently of code coverage, while also discussing the limitations of the analysis [33].

2.9.2 Mutation for JavaScript and web applications

Mutation analysis has been adapted to JavaScript and to web applications, where the cost of generating and running mutants is a central concern. Mirshokraie et al. guide mutant generation with a combination of static and dynamic analysis, directing it at code that is error-prone or likely to affect the output, in order to reduce the cost and the number of equivalent mutants, and they propose mutation operators specific to JavaScript and the web [52]. The journal version adds a ranking of functions and reports that more than 93% of the generated mutants are non-equivalent and that more than 75% of the surviving non-equivalent mutants fall in the top 30% of the ranked functions [53]. Praphamontripong and Offutt define operators for faults in the way web components interact with each other, covering client-to-server and server-to-server interaction, implement them in a tool called webMuJava, and report a small case study suggesting that the resulting tests find web application faults; the work appeared at a workshop [58].

On the tooling side, Stryker is a mutation testing framework for JavaScript and TypeScript. No peer-reviewed publication describing it was found, and the project does not cite one; its documentation describes implementations for JavaScript and TypeScript, for C# and for Scala, more than 30 supported mutations, a runner-agnostic design and an Apache-2.0 licence [66]. The absence of a paper is the reason the mutation procedure in Chapter 4 is specified in the thesis rather than delegated to a tool's defaults.

2.9.3 Reporting a mutation experiment

Papadakis et al. also give best-practice advice on the use of mutation in empirical studies of testing [56]. Chapter 4 follows it by stating the operator set it uses, how the mutants were produced and sampled, which mutants were excluded and why, and the detection rate for each operator separately instead of one aggregate figure. The operators it uses change the navigation structure of the application: renaming a route path, removing a guard, deleting a `routerLink` and retargeting a `Router.navigate` call. The oracle is the difference between the graph extracted before the mutation and the graph extracted after it, compared against the change that was authored, so the expected outcome is known without reference to the tool's own output.

This use of mutation puts the thesis in the change impact analysis tradition rather than in test adequacy measurement. Arnold and Bohner define a framework for comparing impact analysis approaches along three dimensions, namely how an approach is used, how it works internally, and how effective it is [4], and the collection edited by Bohner and Arnold is the standard reference for the area, covering source-code dependency analysis and traceability analysis [12]. Chianti is the classic tool of this kind: it decomposes the difference between two versions of a program into atomic changes and reports, for each affected test, the changes that affect it; on a year of version history from one project it found that an average of 52% of the unit tests were affected, each by only 3.95% of the atomic changes [60]. Lehnert's taxonomy classifies impact analysis approaches along more detailed criteria than the earlier frameworks and is useful for placing a new one, although it appeared at a workshop [43]. A transition graph whose edges carry source witnesses supports the same kind of query at the level of screens, and the mutation experiment measures whether the graph changes when the source it was derived from changes.

2.10 The gap

Navigation models have been extracted statically from web applications, but only from server-rendered frameworks. Client-side routing frameworks have been modelled dynamically, and the only static analysis of such a framework found in the literature excludes routing. Static extraction of a guarded transition graph from client-side router configuration, in which each navigation site is correctly attributed to the screens it can be triggered from, has not been covered by earlier work. The Android literature is the nearest static relative. It attributes navigation by resolving the owning object, and when that fails for shared code it falls back on runtime traces. A navigation site in a web application frequently has no owning object that could be resolved. In Android the owning object of a handler usually identifies the window, so the attribution problem studied here does not arise there in the same form, and an Android technique cannot simply be reused.

The literature reviewed in Sections 2.6 to 2.9 fills in why the problem is open and what is available for addressing it. The analysis machinery is not missing: abstract interpretation of JavaScript is established [29], source-level analysis over a typed syntax tree is a standard tooling style [8, 57], and the Java literature has shown

repeatedly that framework behaviour can be modelled explicitly to recover call edges that a general analysis loses [65, 3, 13]. What has not been done is to apply this to a client-side framework in order to recover navigation. The one peer-reviewed static analysis in the Angular family targets AngularJS 1.x and checks template and controller consistency rather than navigation [55], and the type information that TypeScript adds narrows a dispatch without settling it, because the type system is unsound by design [10, 62]. The reactive literature marks where the boundary of the approach lies, since the firing of a reactive computation is decided by a graph the runtime builds and not by the program text [15, 7]. Mutation testing supplies a method for asking whether an extracted model reacts to source changes [30, 56], and the question of how a statically extracted navigation graph behaves under that method has not been answered for any client-side framework.

Chapter 3

Static navigation analysis and scope attribution

This chapter presents the analysis that is the main contribution of the thesis. Section 3.1 defines the graph model and states the soundness property claimed for it. Section 3.2 presents scope attribution, the analysis proposed in answer to RQ3. Section 3.3 describes how both are implemented in the extractor. The evaluation of the analysis is described separately, in Chapter 4.

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, namely the first and the last component of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge, and the paragraph on witnesses below says how they are traced.*

A *must* edge is claimed to fire for every input on which the enclosing code executes; a *may* edge is possible but conditional; *unknown* marks an edge whose modality could not be determined. The third modality is specified but not produced: no pass implemented in this version assigns *unknown*, every edge in the measured graphs

carries *must* or *may*, and the tables of Chapter 5 report those two tiers only. Guards are stored as source text and are never evaluated by the tool. A consumer of the graph can read a guard, but is not expected to decide whether it holds. This keeps the set G opaque and means that the model does not need to represent application state.

Screens and routes. The set S is derived from the route configuration, and not every route contributes a screen. Write *Routes* for the set of routes the application declares and $component(r)$ for the component that route r renders. Then

$$S \;=\; \{\, screen(r) \;\mid\; r \in \mathit{Routes} \text{ and } component(r) \text{ is defined} \,\},$$

where $screen(r)$ is the node for route r, identified by the full path that concatenates the paths of r and of its ancestors. A route that declares `children` but no `component` is only a path prefix, as Section 2.7 describes: it contributes no element of S, and its path is carried into the full paths of its children. A route whose only effect is a `redirectTo` renders nothing and likewise contributes no element of S. Because $screen(r)$ is identified by the full path, the map $screen(\cdot)$ is injective on the routes whose full paths are pairwise distinct. Angular also permits two component-bearing routes with the same full path, told apart at run time by `pathMatch` or by a `canMatch` guard. The model assumes that this does not occur: the extractor keeps the route declared first under a full path and drops the later ones. Two routes with different paths that render the same component give two screens, which is the case that makes the attribution of Section 3.2 necessary.

Witnesses. Every edge of static or runtime origin carries a witness: the file and line of a construct that produced it, or an observation identifier. Where several constructs produce the same edge, one of them is kept as the witness and the others are recorded as merged witnesses, so that every construct behind an edge can be traced (Section 4.2). No edge is added to the base graph without a witness, and edges added manually are kept in a separate overlay. The tool's specification states this as an invariant, and the first evaluation method checks it mechanically.

Claim 3.2 (Soundness of the must tier)**.** *Every must edge corresponds to a navigation that occurs, for every input on which the enclosing code executes, over the declared route set.*

This is the only soundness property claimed in the thesis, and it holds only relative to the declared route set. No such property is claimed for the may and unknown tiers. The witness of a may edge shows that the corresponding call site exists in the source, but it does not show that the transition is ever taken. This difference between the tiers matters for the evaluation. Whether the target found at a witness location agrees with the target of the edge is a meaningful question for must edges, and for may edges whose target is a literal. It is not a meaningful question for edges that the extractor produced by over-approximation, and treating it as one leads to spurious failures, as described in Section 4.3.2.

3.2 Scope attribution

This section specifies more than the prototype contains, and the division is the same throughout. Component nesting, router outlets, the shell rule and anchored edges are implemented in the extractor and are the parts evaluated in Chapter 4. The call graph with dependency injection, the dispatch relations that carry NgRx effects and router event subscriptions, guards that return a `UrlTree`, navigation sites in interceptors and resolvers, and navigation inside signal effects and observable subscriptions, which the model labels *unknown* (Section 3.2.5), are part of the model only: they are specified here to define the analysis, but no edge in the measured graphs comes from them. The navigation sites they would attribute, in a service, guard, interceptor, resolver or effect, contribute no edge, and the evaluation counts them in the defect class "Attribution, service" of Table 4.2. Section 3.3 gives the same division from the implementation side.

3.2.1 The defect being addressed

In the version of the extractor used for the baseline measurements, a navigation site is attributed to a screen only if it occurs directly in the component that the route for that screen renders. Navigation sites anywhere else are not attributed to any screen. This one cause explains three symptoms in the baseline: shared components contribute no edges, because the application shell places the header next to the router outlet and not inside the subtree of any screen; a component mounted at several routes contributes one edge instead of one per route; and applications that navigate from injected services produce no edges at all.

3.2.2 Scope

Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is thus an element of the powerset $\mathcal{P}(S)$, and the abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y \;=\; X \cup Y,$$

which is the powerset lattice over the screens. Its least element is the empty scope,

$$\bot \;=\; \varnothing,$$

a site that fires from no screen at all. This is the scope of unreached code: the navigation site exists in the source, but no screen has been found from which it can be reached, so it contributes no edge. The greatest element is $\top = S$, every screen.

Three elements of the domain are named, because attribution anchors edges at them. Writing $r' \succeq r$ for the reflexive ancestor relation on routes, that is r' is r itself or an ancestor of r in the route tree, they are

$$\mathit{screen}(r), \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

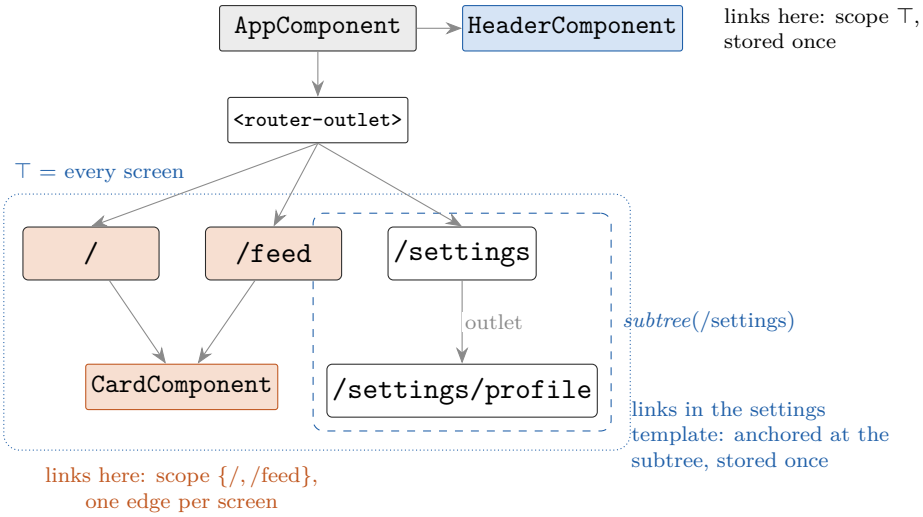


Figure 3.1: Scopes in a small application. The header is rendered next to the outlet, so its links fire from every screen. The card is rendered inside two unrelated screens, so its scope is the union of the two, which is none of the named elements. The settings template is rendered on its own screen and on its child screen, which is exactly *subtree*(/settings). Grey arrows show component nesting and outlets.

and they form the chain

$$\bot \sqsubseteq \mathit{screen}(r) \sqsubseteq \mathit{subtree}(r) \sqsubseteq \top.$$

The chain is stated for a route r with $\mathit{component}(r)$ defined. For a prefix route, which has `children` but no `component`, $\mathit{screen}(r)$ is undefined and only $\mathit{subtree}(r)$, the set of the screens of its descendants, is defined. The chain then reads $\bot \sqsubseteq \mathit{subtree}(r) \sqsubseteq \top$. These names are a representation device for anchoring edges, not a restriction of the domain. The join of two scopes is their union, not the least named element above both: a component mounted at two unrelated leaf routes has the scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, which is none of the three named elements. The names matter for storage, where an edge anchored at $\top$ or at $\mathit{subtree}(r)$ is kept once rather than once per screen. Table 3.1 assigns a scope to each Angular construct. Figure 3.1 shows the three kinds of scope in a small application, and Figure 3.2 shows the domain for three of its screens.

The row for services is why a simple traversal of the component tree is not enough. A service has no scope of its own. Its scope is the join of the scopes of the places that call it, which may themselves be services. Attribution is therefore computed as a fixpoint over the input relations of Section 3.2.3. The outlet relation gives the initial scopes, and scopes are propagated along the component nesting relation N, the call graph CG and the dispatch relation D, with $\mathcal{P}(S)$ as the abstract domain. This is a monotone dataflow problem in the sense of Kildall [37, 54], and two properties of it do separate work. Finite height gives termination. The height of $\mathcal{P}(S)$ is the number of screens $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$, and the iteration can make only finitely many changes. Monotonicity of the transfer functions gives the result: the iteration reaches the least solution of the constraint system, and that solution does not depend on the order in which

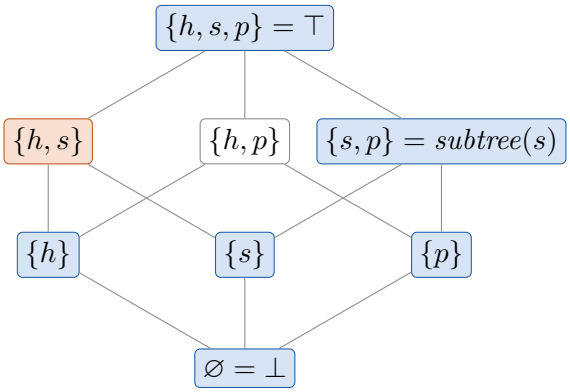


Figure 3.2: The scope domain $\mathcal{P}(S)$ for three screens: h for `/`, s for `/settings` and its child p for `/settings/profile`. Shaded elements are the named scopes at which edges are anchored. The join is set union: a component rendered in both h and s gets $\{h\} \sqcup \{s\} = \{h, s\}$, marked in orange, which is not a named element, so its edges are stored per screen. The height of the lattice is $|S|$.

Table 3.1: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$subtree(r)$
component mounted at routes $r_1, \ldots, r_k$	$subtree(r_1) \sqcup \cdots \sqcup subtree(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
method of an injectable service	join of the scopes of its callers
`canActivate` or `canMatch` on route r	$subtree(r)$
HTTP interceptor	$\top$
NgRx effect for action a	join of the scopes of every dispatcher of a

nodes leave the worklist. The transfer functions here are monotone because the only operation they apply to a scope is a join with another scope, and union is monotone in both arguments.

A worklist solver re-examines a node once for each rise of one of its predecessors, and the work done for a node is proportional to its out-degree in $N \cup CG \cup D$, so a run makes at most

$$O\Big(|S| \cdot (|N| + |CG| + |D|)\Big)$$

transfer-function applications. Each application is a union of two subsets of S, which costs $O(|S|)$ with scopes held as bit sets, so the running time is bounded by $O(|S|^2 \cdot (|N| + |CG| + |D|))$. This is the bound for the full model.

The part of the analysis that is implemented, the nesting and outlet relations, is the full model with CG and D empty, so the bound above specialises to it. The implemented pass is cheaper than that specialisation, because reachability from

each source needs no set-valued joins. Scopes are propagated along the component nesting graph from the shell and from the component of each screen, which is plain reachability and costs

$$O\Big(|S| \cdot (|C| + |N|)\Big),$$

one traversal of the nesting graph per screen, where C is the set of components. What makes the propagation necessary in both cases is that scopes travel transitively and the scope of one node can rise more than once, once for each screen that reaches it. Cycles are not the reason. The nesting and outlet relations the prototype implements may contain cycles, since mutually recursive templates are legal, and the worklist handles them; a cycle in CG or D likewise only means that a node can be re-examined after one of its own successors.

3.2.3 Input relations

Four relations, in order of increasing difficulty, provide the input to the fixpoint computation. *Component nesting*: $A \to B$ when A's template instantiates B's selector, resolved within the correct import scope; matching selector text globally is unsound, since two components may share a selector string in different scopes. *Router outlets*: components on the path from the root to the outlet form the shell and take $\top$; components below the component of route r take its scope $\mathit{subtree}(r)$; a named outlet carries a second mount, which makes auxiliary routes representable without separate treatment. *Call graph with dependency injection*: a call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17], with Angular's provider declarations as the source of bindings, as recommended for framework-heavy code [3]. Where a token admits several implementations the analysis joins over all of them, which is one source of *may* modality. *Dispatch relations*: a dispatch is treated as a call-graph edge from every dispatcher to every handler, which admits NgRx effects and router event subscriptions under the same fixpoint.

Algorithm 1 gives the computation, and its propagation steps are the transfer-function applications bounded above. A step adds a screen to a scope only when an edge of N, CG or D carries it there, and scopes only grow, so what the algorithm computes is the least solution of the constraint system that Table 3.1 states. In the model a component that is both mounted by a route and rendered in the template of another component therefore holds the join of the two scopes. Section 3.3 reports that the prototype does not propagate into a component that a route mounts, so it computes the least solution of a restricted system rather than of this one.

3.2.4 Guards as program points

A guard that returns a `UrlTree` is itself a navigation: a redirect edge from $\mathit{subtree}(r)$, typically to a login screen. Treating guards only as labels on inbound edges misses this edge class entirely. Further, the guard of an edge is the conjunction of the conditions on the containment path from screen to call site (structural directives) with the guards of the target route. Reporting only the second part gives a weaker

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$. The loop marked *model only* belongs to the full analysis and is not implemented in the prototype evaluated in Chapter 5.

Require: routes R with route tree; components C; nesting $N \subseteq C \times C$; call graph CG; dispatch relation D
Ensure: $scope(x) \in \mathcal{P}(S)$ for every component and callable x

for all x **do** $scope(x) \leftarrow \varnothing$
end for
for all $r \in R$ with $component(r) = c$ **do**
    $scope(c) \leftarrow scope(c) \cup subtree(r)$ ▷ own screen and outlet descendants
end for
for all non-routed c holding the primary outlet, not nested in any component **do**
    $scope(c) \leftarrow S$ ▷ application shell, $\top$
end for
$W \leftarrow$ every x with $scope(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        PROPAGATE(x, y)
    end for
    for all y with $(x, y) \in CG \cup D$ **do** ▷ *model only*: DI calls, dispatch
        PROPAGATE(x, y)
    end for
end while
procedure PROPAGATE(x, y)
    if $scope(x) \not\subseteq scope(y)$ **then**
        $scope(y) \leftarrow scope(y) \cup scope(x)$; add y to W
    end if
end procedure

condition than the true one. In the baseline, for example, a link that is rendered only inside `@if (isUser())` was labelled with the authentication guard of its target route. That guard is satisfied by any logged-in user, including users who are not the owner and who therefore never see the link.

3.2.5 Boundary

A navigation inside a signal effect has a firing condition that is a data dependence on a reactive graph. As Section 2.7 notes, only the read set is statically visible. The model gives such edges the modality *unknown* and records the set of signals read by the effect as the symbolic guard; no pass in this version emits one. The analysis does not try to determine when these edges fire. It marks this limit explicitly instead of giving a result that looks more precise than it is.

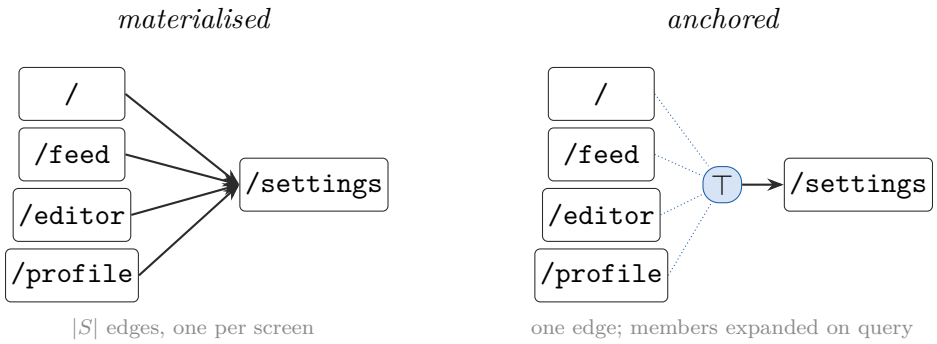


Figure 3.3: The same header link stored two ways. Materialised, it becomes one edge per screen. Anchored, it is stored once at the scope node $\top$, and *edgesFrom*(r) adds it to the edges of any screen r when asked. Claim 3.3 states that both forms describe the same transitions.

3.2.6 Anchored edges

If the result of attribution were stored explicitly, a single link in the application shell would produce one edge for every screen, and the size of the graph would be bounded by $|S| \cdot |\mathit{sites}|$, where *sites* is the set of navigation sites. For the largest application in the corpus this bound is 610×1344. Instead, each edge is stored once, anchored at its scope, and the set of edges leaving a particular screen is computed when it is requested:

$$\mathit{edgesFrom}(r) \;=\; E_{\top} \;\cup \bigcup_{r' \succeq r} E_{\mathit{subtree}(r')} \;\cup\; E_{\mathit{screen}(r)},$$

where E_X is the set of edges anchored at scope X and $r' \succeq r$ is the reflexive ancestor relation on routes introduced above, so that the term for $r' = r$ contributes $E_{\mathit{subtree}(r)}$ itself. The query is made for a screen, so $\mathit{component}(r)$ is defined, while r' ranges over all ancestors of r, prefix routes included. An edge whose scope is some other screen set is stored at each of its screens and reaches *edgesFrom*(r) through the $E_{\mathit{screen}(r)}$ term. Figure 3.3 contrasts the two representations.

Claim 3.3 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set, modulo the declared approximation list of Section 6.3.*

3.3 Implementation

The extractor, *uigraph*, is available as open source. It consists of a core that is independent of any particular framework and a set of adapters, one per framework. The core defines the graph model, its representation on disk, and the operations over graphs. Each adapter maps the source constructs of one framework onto the shared model, following a fixed interface. The Angular adapter reads route configuration, templates and class code with the TypeScript compiler API, and emits nodes for screens and edges for `routerLink` attributes and `Router.navigate` calls, each with the rule identifier and source location that produced it as witness.

Scope attribution is implemented as a separate pass in the Angular adapter, enabled with the option `-attribution`. With the option off, the extractor produces exactly the baseline edge set; this was checked on every application in the corpus. The pass implements two of the four input relations of Section 3.2.

Component nesting. Every component in the application, excluding test hosts and stories, is read with its element selectors and its template. An edge $A \to B$ is added when A's template contains an element with B's selector. When a selector belongs to more than one component, the candidates are narrowed to those that A's file imports; if none is imported, the analysis joins over all of them and records a note. Attribute selectors are not matched.

Router outlets. The route tree is recorded during route extraction. A route component's template is rendered on its own screen and on every descendant screen, so its navigation sites get the scope $\mathit{subtree}(r)$. A component that is not routed, is not referenced by any template and holds the primary `<router-outlet>` is the application shell, and everything it renders gets the scope $\top$. A component that is not routed and not referenced, but whose subtree contains the outlet, is most likely mounted by a route the extractor cannot see, for example through a helper function that wraps a route table. It is also treated as shell, but its edges are capped at *may*, so that Claim 3.2 is not put at risk by a scope that may be too large.

The pass departs from Algorithm 1 in one respect. It does not propagate a scope into a component that a route mounts: a routed component keeps $\mathit{subtree}(r)$ and does not also receive the scope of a component whose template renders it. The prototype therefore computes the least solution of the system of Table 3.1 with the template row restricted to targets that no route mounts. The result is contained in the solution the full system would give, so a navigation site in such a component can be attributed to fewer screens than the model allows, never to more. A scope that is too small loses edges and cannot invent them, which is why the restriction cannot put Claim 3.2 at risk; what it costs is recall, and the constructs it strands are counted in Chapter 5.

Anchored edges. Edges with scope $\top$ are stored once at a node of kind `scope`. Edges whose screen set is exactly $\mathit{subtree}(r)$ are stored once at a scope node that lists the members of that subtree. Other screen sets are stored as one edge per screen. The core library provides the expansion query $\mathit{edgesFrom}(r)$ and a function that materialises all anchored edges, which is used to test Claim 3.3.

The call graph with dependency injection and the dispatch relations are not implemented in this version. Navigation sites in services, guards, interceptors, resolvers and effects are not attributed to any screen, and the evaluation counts them in the defect class "Attribution, service" of Table 4.2. The pass is covered by unit tests for each of the cases above, including a test that the baseline is unchanged when the option is off.

One change was made to the baseline itself. For components with an inline template, the witness of an edge pointed at the line where the template starts rather than at the construct. The witness now points at the construct's own line. This can change witness locations but no edge. On the corpus, the baseline graphs with and without this change are identical, including witnesses.

Chapter 4

Evaluation setup and research material

This chapter describes how the analysis of Chapter 3 is evaluated. Section 4.1 presents the corpus of applications and discusses why it can be considered a valid basis for the measurements. Section 4.2 defines the metrics used to answer each research question. Section 4.3 describes the four evaluation methods, what each of them can and cannot show, and the arrangements that keep the results reproducible.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria. The criteria were applied in order of increasing cost, so that a candidate could be rejected cheaply before more expensive checks were made. The criteria were: (1) a real application rather than a library or template; (2) a nested, lazily-loaded route tree; (3) guards present, since guards are what produce conditional transitions; (4) an end-to-end test suite that drives a browser; (5) retrievable history in which tests and source change together; (6) an active repository. Six candidates satisfied all six criteria and were retained (Table 4.1). Each of them is fixed to a specific commit so that the measurements can be repeated on exactly the same code.

Four candidates were rejected. They are listed here together with the reason for rejection, since the rejected candidates are part of the selection protocol. `angular/components` is a component library with no product router. `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none of which drive the user interface. `gravitee` declares 618 routes but has 14 UI specifications. `SAP/spartacus` has a large Cypress suite, but its routing is configuration-driven in a way that the extractor does not currently support, so it was set aside as a possible candidate for later work.

The screening also produced an observation that influenced the design of the evaluation. Among the candidates examined, the size of the route tree and the density of end-to-end tests were inversely related. The largest applications had few or no tests that exercise the user interface, while the application with the most extensive navigation-oriented test suite was of medium size. As a result, no single application

Table 4.1: The corpus. Routes are the route entries reached by the independent route parser of Section 4.3. For kubernetes/dashboard the parser cannot follow 37 lazily loaded entries, so its count is a lower bound.

Application	Commit	Routes	Role in the study
gothinkster/angular-realworld-example-app	`dd99ed2c`	14	size floor; all literal paths
alfresco/alfresco-content-app	`b88d47f4`	55	87 Playwright specs including a navigation suite
ghostfolio/ghostfolio	`9884fa0b`	135	constant-valued route paths; self-hostable
kubernetes/dashboard	`0ba796d9`	$\geq$42	mid-size control
bitwarden/clients (web)	`e40ce6f0`	175	highest guard density
openMF/web-app	`7f781796`	921	size ceiling

in the corpus provides both a large route tree and execution-based ground truth. The evaluation therefore applies the methods that require test suites only to the applications that have them, and the methods that do not require tests to all six.

4.1.2 Validity of an open-source corpus

Open-source corpora are common in empirical software engineering, and their threats to validity have been studied. Kalliamvakou et al. show that a large share of GitHub repositories are inactive, personal or not software projects, that stars do not measure quality, and that forks inflate counts [34, 35]. The remedy they recommend is to filter the candidates using explicit criteria and to report those criteria, which is what the selection procedure above does. The threat that remains is one of external validity. Open-source applications may differ from proprietary enterprise applications in size, in routing style and in the mechanisms used for navigation. In addition, I selected the corpus and I wrote the extractor, which introduces a possible selection bias.

Three measures address these threats. The selection protocol is reported in full, including the rejected candidates, and the criteria were fixed before any results were obtained. The corpus is also stratified across two orders of magnitude in the number of routes, so that any effect of application size is visible and not averaged away. Finally, the commits are fixed and the evaluation harness is publicly available. A fourth measure is planned: a case study on a proprietary Angular application from industry, reporting only aggregate figures; this would be the most direct response to the external validity threat, and it is discussed further in Chapter 6. There is also no standard benchmark corpus for single-page applications. The web testing survey confirms that evaluations in this area are carried out on collections of applications chosen for each study [36]. For this reason the corpus and its manifest are reported

as one of the outputs of the thesis.

4.2 Metrics

Each research question is answered with one main measure. They are defined here.

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code, in the application source excluding tests. Constructs are found by an independent parser that shares no code with the extractor. Attributes such as `routerLinkActive` are not constructs; counting them by substring was one of the corrections described in Section 4.3.2.

Conversion ratio (RQ1). The conversion ratio is the fraction of navigation constructs that are witnessed by at least one must edge. When several constructs produce the same edge, the extractor keeps the first as the edge's witness and records the others as merged witnesses, so every construct that yields an edge can be traced. The ratio is computed over constructs rather than edges because the number of edges depends on representation: the same construct can be stored as one anchored edge or as one edge per screen (Section 3.2). An earlier version of this measure divided the number of must edges by the number of constructs. It was replaced for that reason (Section 4.3.2).

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct with no such edge is *lost*, and it is assigned to exactly one defect class by the first rule in Table 4.2 that applies. Because the rules are ordered, the classes partition the lost constructs and their sizes can be compared within each application. Guard defects do not appear in this table, since a wrong guard mislabels an edge rather than removing it; they are measured by the witness check.

The nested branch decides a class from the state of the construct's *owning* route component. A construct can therefore be assigned to one of the node classes and still be converted once attribution gives it a scope through the shell or a subtree, because the scope it needs is not the one its owner would have supplied. Section 5.3 measures how often that happens.

A call of the form `navigate([], {queryParams})` changes only the query string of the current screen. It is not a transition between screens, so it is reported separately and not counted as a loss.

Change in coverage and must-tier precision (RQ3). The conversion ratio and the loss breakdown are measured on one commit of the extractor, run twice on identical inputs with the attribution pass disabled and enabled. Measuring one commit rather than two rules out any difference between the runs other than the pass itself, which two versions could not guarantee; Section 3.3 reports that with the pass off the extractor reproduces the baseline edge set exactly on every application. *Must-tier precision* is the fraction of must edges whose witness line contains a construct of the claimed kind with the claimed target. RQ3 is answered positively only if coverage rises and must-tier precision does not fall.

Table 4.2: Rules that assign a lost construct to a defect class, in the order the harness tests them. The indented rules are the nested decision taken when the construct's owning route component has no node.

Class	The construct ...
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

Node-level recall. The fraction of distinct URLs visited by an application's own passing end-to-end tests that have a matching node in the graph. It is a cross-check on node extraction, which all edges depend on.

Detection rate (RQ4). The fraction of injected source changes that produce the predicted change in the graph, reported per mutation operator with its n. A change whose construct the graph does not contain is excluded rather than counted as a miss, since it can only repeat a coverage gap measured under RQ1. The rate is therefore conditional on coverage, and the number of excluded mutants is reported alongside it.

4.3 Evaluation methods

Four methods are used. None establishes correctness alone, so Table 4.3 states what each can show and where it is blind. Figure 4.1 shows which part of the graph each method checks and where its ground truth comes from.

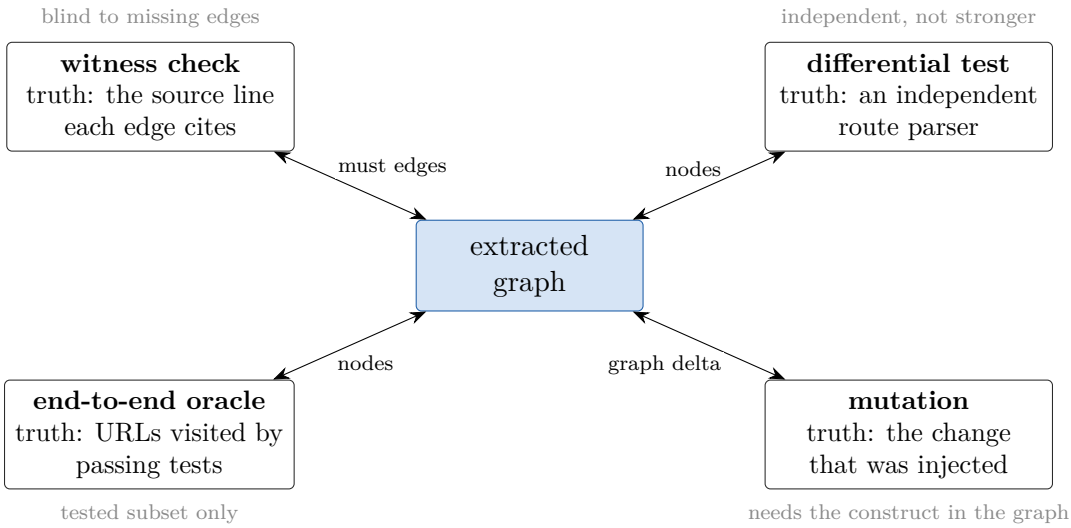


Figure 4.1: The four evaluation methods, the part of the graph each one checks, and where its ground truth comes from. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.3.

4.3.1 Preconditions

Two properties of the extractor were checked before any of the measurements were taken. The first is determinism: two extractions from the same unchanged source produce identical sets of nodes and edges, when compared on identifier, route, event, guard and modality after sorting. Without this property the differences observed in mutation testing could not be distinguished from noise. The second is the absence of accumulated state: extracting into an existing database gives the same graph as extracting after the database has been deleted. Without this property the results for a mutant could be affected by the earlier baseline run.

4.3.2 Corrections made to the protocol

Seven corrections were made to the protocol while it was being developed. They are described here because each of them would otherwise have led to an incorrect figure in this thesis, and because the underlying mistakes are general ones.

In the end-to-end oracle, a URL parameter observed in a test was at first allowed to match a literal segment of a route, which produced matches that did not correspond to real screens. After the rule was tightened, the recall figure for one application fell from 86.7% to 66.7%. A negative test, which visits a URL that must not exist, is now also excluded, which gives the 64.3% reported in Chapter 5.

The first version of the witness check reported a precision of 44.7% for one application. On inspection, all 177 edges it had flagged were may edges with a confidence of 0.5 that referred to a single source line. They were the result of one over-approximated call site being connected to every declared route. The must tier of the same run had 69 correct edges out of 69.

In mutation testing, a mutant whose target does not appear in the baseline graph

Table 4.3: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-tier precision. For every must edge, the witness file is re-parsed at the claimed line and the construct kind and target are compared.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists.
Differential test	Disagreements with an independently written route-configuration parser that shares no code with the extractor.	Diagnostic, not a metric. The parser is independent, not stronger: its first version missed whole route tables in several applications until it was corrected (Section 4.3.2).
End-to-end oracle	Node-level recall. Every URL a passing test visits is a screen that provably exists; ground truth comes from execution.	Tested subset only. A direct `goto` is not a UI transition, so visit pairs are not counted as edges. Textual URL extraction under-samples click-driven navigation.
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta, so nothing is circular. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a coverage gap and is excluded, as is a guard mutant on a screen that no edge enters. Detection is therefore conditional on coverage. Sampling is seeded and stratified by operator.

cannot show anything about the graph's sensitivity to change; it only repeats a coverage gap that has already been counted elsewhere. Three of the five mutation misses for one application were of this kind and were moved to a separate category.

The first denominator of the conversion ratio counted the substring `routerLink` in templates, which also matches the attributes `routerLinkActive` and `routerLinkActiveOptions`. For alfresco this counted 113 constructs where the parser finds 34. Constructs are now counted by the parser that also produces the loss breakdown, so the ratio and the breakdown share one definition.

The first version of the route parser used by the differential test did not look inside wrapper calls such as `Route.withShell([...])`, did not follow a lazily loaded NgModule to the routing module it imports, and walked lazily loaded route tables a second time without their prefix. On mifos it therefore reported that every declared screen had a node. After the correction it finds 610 declared screens, of which 58 have a node. The extractor was the same in both runs.

The conversion ratio itself was first defined as the number of must edges divided by the number of constructs. After attribution, this ratio rose from 5.1% to 75.7% for mifos, because a construct in a component rendered on many screens produced one edge per screen. So the rise came from how edges were stored. It is now computed per construct, and edges whose screen set is a whole subtree are stored once (Section 3.2).

The mutation harness patched the source by searching for the text of the construct, so a mutant aimed at one of several identical constructs edited the first of them instead of its own. In realworld all five guard mutants removed the same guard. The harness also judged detection on the node and edge sets alone, which scored the deletion of one of two identical links in a template as a miss, although the graph had behaved correctly: it kept the edge, which the second link still produces, and dropped only the witness that pointed at the deleted line. Detection is now judged on the witness set as well, which is the property change impact analysis actually needs. Three smaller changes followed from the same review. A retargeting mutant now points at another declared route instead of an invented one, so that it cannot be confused with a missing target. A guard mutant on a screen that no edge enters is excluded, because a guard is represented on the edges entering the screen it guards, so such a mutant can only repeat the edge-coverage gap already counted under RQ1. A rename mutant counts as detected when the renamed route appears in the graph, without also requiring the original node to disappear, since the same path segment may be declared in a second routing module. With the offset and witness corrections in place, on the same seed and the same sample, the detection rate for realworld changed from 71.4% to 95.2%. Sampling is seeded and stratified with a fixed quota per operator, and the detection rate is reported per operator with its n.

Five of the seven corrections were found the same way, by a figure that looked too good or too bad to believe: the conversion ratio that rose to 75.7%, the precision that fell to 44.7%, the coverage that reached 100%, the construct count that tripled, and the detection rate that moved by more than twenty points. Each turned out to be a property of the measurement rather than of the extractor. A figure that is surprising is therefore checked against its inputs before it is written down, which is how the remaining figures in this thesis were arrived at.

4.3.3 Reproducibility

The whole evaluation can be re-run with a single command. The command clones the extractor at the commit tagged `thesis-eval`, fetches each application in the corpus at its fixed commit rather than from a branch, runs the extraction and all four evaluation methods, and writes a results file together with the tables derived from it. Each figure in the results file is recorded together with the commit of the extractor, the commits of the corpus applications, the version of the harness, and the version of the runtime. The tables in this thesis are generated from the results file and not entered by hand.

Chapter 5

Results

The figures in this chapter were produced by the harness of Section 4.3 on the corpus of Section 4.1. Every table is generated from the harness's results files. The baseline is the extractor without scope attribution; the attributed version is the same code with the attribution pass enabled.

5.1 Baseline extraction (RQ1)

Table 5.1 gives the conversion ratio for each application. The baseline recovers between 0% and 15% of the navigation constructs as must edges. Two applications, ghostfolio and k8s dashboard, yield no edges at all, and alfresco yields one. Edge counts must be read per tier. The 177 may edges of mifos are a single over-approximation from one unresolvable call site, not 177 transitions.

A transition can only start at a screen that exists in the graph, so node extraction was measured as well. Table 5.2 and Figure 5.1 compare the nodes with the screens found by the independent route parser. Node extraction is complete on realworld, at 10 declared screens of 10, and nearly complete on alfresco, at 24 of 25. It is far from complete on the larger ones: about half of the declared screens of bitwarden and under a tenth of those of mifos have a node, and ghostfolio has a single node for 84 declared screens. For k8s dashboard the parser itself cannot follow the lazy route tables, so its 100% covers only two screens and says little.

So the baseline recovers a small fraction of the navigation, and loses it at two levels: some screens never become nodes, and most navigation sites of the screens that do are not turned into edges.

5.2 Defect classes (RQ2)

Table 5.3 lists the defect classes and the independent methods that confirm each. The five classes found in the first round all have at least two confirmations. The quantitative breakdown below revealed a sixth candidate, a node that exists but is not linked to the component that renders it. It is confirmed only by the comparison between the route parser and the graph, so it is reported as a candidate until a

Table 5.1: Construct-to-transition conversion per application, baseline. Conversion is the fraction of constructs witnessed by a must edge; the edge columns count edges.

Application	Constructs	Nodes	Must	May	Conversion
realworld	40	11	7	14	15.0%
alfresco	34	30	1	0	2.9%
ghostfolio	198	1	0	0	0.0%
k8s dashboard	72	40	0	0	0.0%
bitwarden	132	78	10	10	12.1%
mifos	1344	178	69	177	5.4%

Table 5.2: Node coverage: declared screens that have a matching node in the graph.

Application	Declared screens	Matched by a node	Coverage	Parser blind spots
realworld	10	10	100.0%	0
alfresco	25	24	96.0%	3
ghostfolio	84	0	0.0%	0
k8s dashboard	2	2	100.0%	37
bitwarden	97	47	48.4%	5
mifos	610	58	9.5%	1

second method confirms it.

Table 5.4 and Figure 5.2 assign every navigation construct to one outcome using the rules of Table 4.2. Of the 1820 constructs in the corpus, 96 are witnessed by must edges and 22 by may edges. Guard defects are measured separately (Section 4.2).

Attribution is the largest class in four of the six applications. In realworld it accounts for 22 of 23 lost constructs, in k8s dashboard for 56 of 68, in bitwarden for 56 of 89 and in alfresco for 18 of 33. Most of it comes from shared components rather than from services.

The other two applications are dominated by node defects. In ghostfolio, 107 of 191 lost constructs sit in screens whose route path is a constant reference, which is why the graph collapses to one node. In mifos, 607 lost constructs sit in lazily loaded screens and 378 in screens whose node has no component link. Because mifos is by far the largest application, these two classes also dominate the corpus total, so the per-application view and the total give different rankings.

The classes also interact, and the interaction runs both ways. A construct in a shared component that is only used inside screens which have no node is counted as attribution, although attribution alone cannot recover it. In the other direction, a construct whose own route component has no node is counted as a node defect, yet attribution can still reach it through a shell or subtree scope and convert it. The 468 attribution constructs are 28% of all lost constructs, and they are therefore neither a floor nor a ceiling on what scope attribution recovers. Section 5.3 measures both directions.

Table 5.5 reports the witness check and the end-to-end oracle. On the must tier, 86 of 87 edges are clean. The one failure is a must edge whose witness navigates to

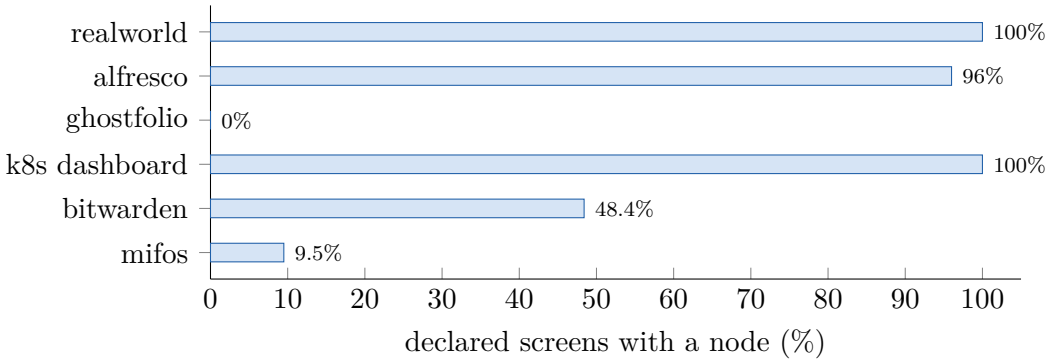


Figure 5.1: Node coverage, baseline: the share of declared screens that have a node in the graph. The value for k8s dashboard rests on two screens only, because the route parser cannot follow its lazily loaded route tables.

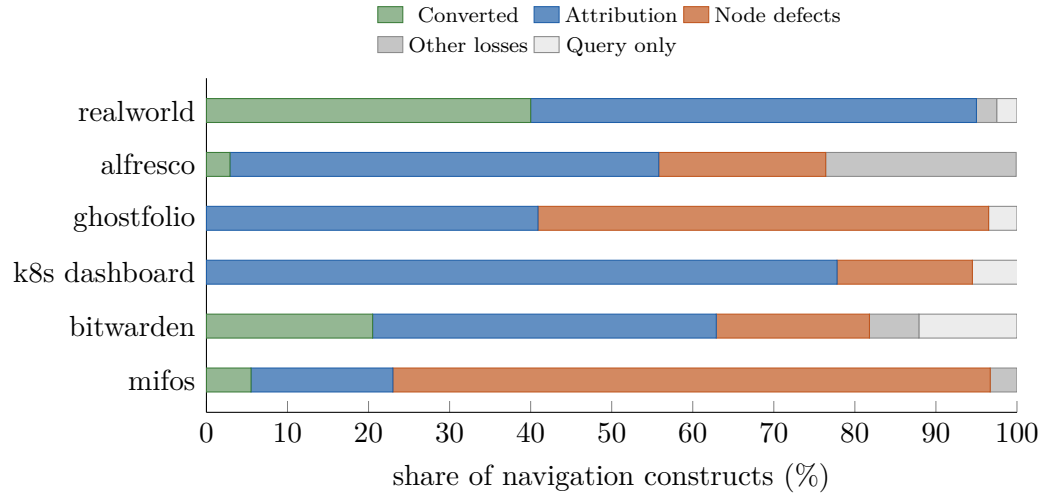


Figure 5.2: What happens to the navigation constructs of each application in the baseline. Attribution losses dominate in four applications; node defects dominate in ghostfolio and mifos. The exact counts per class are in Table 5.4.

`/profile/:username` while the edge targets `/profile`; it violates Claim 3.2.

5.3 After attribution (RQ3)

Table 5.6 and Figure 5.3 compare the extractor with and without scope attribution on identical inputs. The comparison is made construct by construct: a construct is *recovered* if it was lost to attribution in the baseline and is witnessed by an edge after attribution.

Coverage rises in four of the six applications. The largest change is on realworld, from 15.0% to 37.5%, where 21 of the 22 attribution losses are recovered; the one left is a navigation inside a service, which this version does not handle. On mifos the ratio almost doubles, from 5.4% to 10.0%. On bitwarden and k8s dashboard it rises by 3.8 and 4.2 percentage points. On alfresco and ghostfolio nothing changes.

Table 5.3: Defect classes and the independent methods confirming each. A class is admitted only with two confirmations. This summary is compiled by hand from the evaluation methods; it is not harness output.

Class	Independent confirmations	Count
Attribution: navigation outside route components is invisible	mutation misses; multi-mount edge absent; manual audit; zero-edge applications	4
Lazy module boundary not followed	route parser; end-to-end oracle; mutation	3
Guards unreliable in both directions	removal invisible to mutation; witness check finds wrong predicate and colliding identifiers	2
Named router outlets not modelled	end-to-end oracle; source survey	2
Constant-valued route paths not resolved	route parser; node-count collapse	2
Candidate: node not linked to its component	route parser against graph	1

Precision on the must tier does not fall. After attribution, 155 of 156 must edges pass the witness check, and the one failure is the realworld edge that already failed in the baseline. No construct that was converted in the baseline is lost after attribution: following every construct by its identifier across the two runs finds no construct that leaves the converted classes.

The witness check also records the edges outside the must tier. Chapter 3 excludes them from the precision claim, because target agreement is not a meaningful test for an edge the extractor produced by over-approximation, and the counts are given here so that the exclusion is visible rather than assumed. On the four applications whose per-edge records were kept, 179 of 201 non-must edges fail the target comparison before attribution and 553 of 599 after it. Mifos accounts for 177 and 529 of those, which is the single over-approximated call site of Section 4.3.2 seen one edge at a time. The growth follows the growth of the may tier and says nothing about the must tier.

The anchored representation was checked on the same graphs. Claim 3.3 requires

Table 5.4: Outcome of every navigation construct, baseline. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	6	1	0	0	16	73	96
Converted, may	10	0	0	0	11	1	22
Attribution, shared component	21	10	72	51	41	225	420
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	3	607	610
Constant path	0	0	107	0	19	0	126
Component unlinked	0	6	0	2	3	378	389
Named outlet	0	7	0	0	0	0	7
Constant target	0	1	0	0	2	16	19
Node missing, other	0	1	3	10	0	6	20
Other	1	0	0	0	6	28	35
Query only (not a transition)	1	0	7	4	16	0	28
All constructs	40	34	198	72	132	1344	1820

Table 5.5: Results of the witness check and the end-to-end oracle, baseline. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

Method	Result	n
Witness check, must tier	86 of 87 clean; one wrong target in realworld.	87
End-to-end oracle, alfresco	9 of 14 visited URLs have a node; one negative test excluded. Two of the five misses are named-outlet URLs.	14

the expansion of the anchored edge set to equal the materialised edge set. On realworld the attributed graph stores 50 edges against two scope nodes; expanding them screen by screen through the query of Section 3.2 yields 97 edges, materialising the whole graph yields 97, and the two sets are identical. On alfresco the attributed graph has no scope node, so the check holds trivially and says nothing. The unit tests of the attribution pass cover the same property on fixtures. The claim is therefore supported on one application of the corpus and on fixtures, not across the corpus, and the 50 against 97 is also the only measurement of what the anchored representation saves.

Across the corpus, attribution converts 81 of the 468 constructs that the loss rules assign to it, 38 of them into must edges and 43 into may edges. It also converts 43 constructs that the rules assign to node defects, 41 of those into must edges. Table 5.7 gives the breakdown by the class each construct held in the baseline. Deterministic conversions over the whole corpus rise from 96 to 175, so of the 79 constructs gained on the must tier, 38 come from the attribution classes and 41 from the node classes. The classes are not independent, and the attribution class is not the ceiling that Section 5.2 might suggest.

Table 5.6: Before and after scope attribution. Conversion in percent and the change in percentage points; must-tier edges that pass the witness check; constructs that the loss rules assigned to attribution in the baseline, and how many of them converted into must and into may edges. Conversions of constructs assigned to other classes are in Table 5.7.

	Conversion			Must-tier clean		Attribution losses, recovered		
Application	Before	After	pp	Before	After	Before	must	may
realworld	15.0%	37.5%	+22.5	6/7	9/10	22	9	12
alfresco	2.9%	2.9%	+0.0	1/1	1/1	18	0	0
ghostfolio	0.0%	0.0%	+0.0	–	–	81	0	0
k8s dashboard	0.0%	4.2%	+4.2	–	3/3	56	2	0
bitwarden	12.1%	15.9%	+3.8	10/10	13/13	56	5	1
mifos	5.4%	10.0%	+4.6	69/69	129/129	235	22	30
All						468	38	43

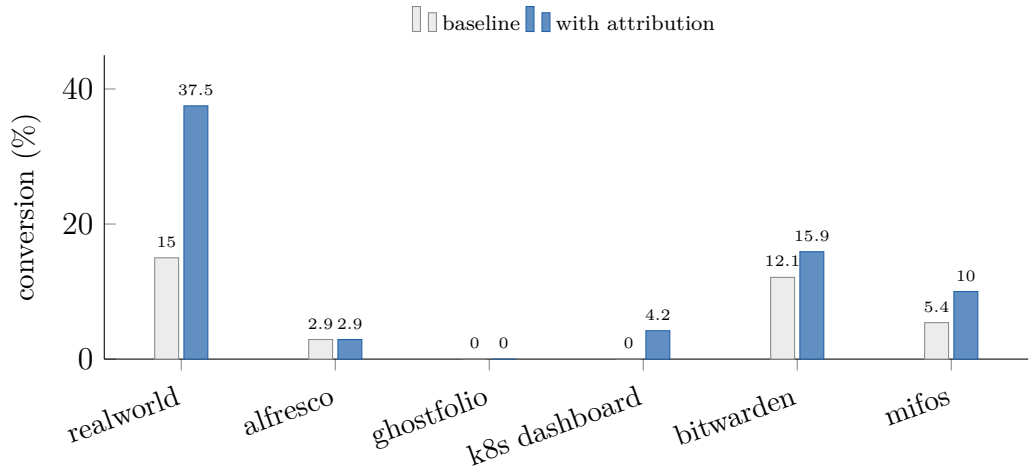


Figure 5.3: Conversion ratio before and after scope attribution, on identical inputs.

Table 5.8 classifies each construct that stays lost.

Four reasons account for most of the gap. In 156 cases the component holding the navigation is never reached from any screen. A rough check of these components suggests three causes: about 60 are opened programmatically, most as dialogs; about 70 are used only inside other components that are themselves unreached; and about 20 have a selector that no application template uses. In 132 cases the scope is found but the target screen has no node, so there is nothing to draw an edge to. This is the node defect of RQ1 seen from the other end. A further 48 constructs are in services and guards, which this version leaves out, and 34 have a fully dynamic target. The remaining 17 are not yet explained and will be inspected by hand.

Scope attribution raises coverage without lowering must-tier precision. On realworld it recovers 21 of its 22 attribution losses, and realworld is the only application whose node extraction is complete and whose stranded navigation sites sit in components that a screen renders. Complete node extraction is not sufficient on its own: alfresco is at 24 declared screens of 25 and recovers nothing, because its 18 remaining losses are 10 sites in components that no screen reaches and 8 in services, and this

Table 5.7: Where the converted constructs came from. Each row is the class the loss rules assigned to the construct in the baseline; the columns give the tier it converted into after attribution. Constructs are followed by identity, not matched by source location.

Baseline class of the construct	must	may	total
Attribution, shared component	38	43	81
Lazy boundary	40	2	42
Component unlinked	1	0	1
Out of the attribution classes	38	43	81
Out of the node classes	41	2	43
All conversions gained	79	45	124

Table 5.8: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	10	31	20	24	71	156
In a service or guard (not implemented)	1	8	9	5	15	10	48
Scope found, target screen has no node	0	0	32	3	2	95	132
Scope found, target fully dynamic	0	0	2	26	1	5	34
Scope found, not explained	0	0	7	0	8	2	17

version attributes neither. On the larger applications the gain is limited by defects in node extraction, which attribution does not address.

The bottleneck these measurements identify is therefore node extraction rather than attribution. In ghostfolio every route path is a reference into a constant table, written as `path: publicRoutes.about.path` over the tables `publicRoutes` and `internalRoutes`. The extractor does not follow those references, so the graph holds a single node that matches none of the 84 declared screens, and no navigation site in the application has a screen to start from or to reach. In mifos two defects compound: 607 lost constructs sit behind lazy module boundaries that the extractor does not follow, and a further 378 sit in screens whose node exists but carries no link to the component that renders it. In both applications attribution behaves as designed and still returns nothing, because it resolves the scope of a navigation site and then finds no target node to draw an edge to. This is the shape of 132 of the 387 attribution losses that survive attribution. An edge cannot be drawn to a screen that was never created, so the ceiling on what attribution can contribute to these applications is set by node extraction and not by attribution itself.

5.4 Change sensitivity (RQ4)

Mutation testing requires the application to be extracted again once per mutant, so this part of the evaluation was run on three of the six applications: realworld and alfresco, which carry the earlier mutation figures, and bitwarden, which has the

Table 5.9: Mutation sample. Candidates found, mutants sampled under the per-operator quota, and mutants excluded as confounded with a coverage gap.

Application	Candidates	Sampled	Excluded
realworld	22	22	1
alfresco	25	16	5
bitwarden	202	32	16

Table 5.10: Detection rate per mutation operator on the attributed graph, as detected over scored mutants. A dash means the operator found no candidate in that application.

Operator	realworld	alfresco	bitwarden
Rename a route path	6/6	8/8	8/8
Delete a `routerLink`	6/6	–	0/0
Retarget a `navigate`	4/4	1/1	2/3
Remove a guard	4/5	0/2	0/5
All operators	20/21	9/11	10/16
Detection rate	95.2%	81.8%	62.5%

highest guard density in the corpus. Every figure in this section comes from the attributed graph with harness 1.3.0, a fixed seed and a quota of eight mutants per operator. The corrections made to the mutation harness are described in Section 4.3.2.

Table 5.9 shows how the sample was drawn. Over the three applications the harness found 249 candidate mutations and scored 48 of them. Twenty-two were excluded, either because the construct a mutant changes is absent from the graph, or, for a guard, because no edge enters the screen that guard protects. An excluded mutant carries no information about sensitivity, since it can only repeat a coverage gap that RQ1 has already counted. Where the quota capped an operator, the candidates left unsampled are reported rather than passed over.

Table 5.10 gives the detection rate per operator, and Figure 5.4 shows the same outcomes summed over the applications. Renaming a route path was detected in all 22 scored mutants of the three applications. Deleting a `routerLink` was detected in all six scored mutants of realworld, and in four of those six the change appeared in the witness set rather than the edge set, because a second identical link in the same template still produces the edge. That is the behaviour the graph should have, and it is the reason detection is judged on witnesses as well as edges. The operator found no candidates in alfresco, which binds `[routerLink]` instead of writing a literal attribute, and in bitwarden its eight sampled candidates were all excluded. Retargeting a `navigate` call was detected in seven of its eight scored mutants. The exception is in bitwarden, where the mutant added the expected edge to the new target but removed no edge to the old one, because the baseline had not resolved that call site to the old target in the first place.

The rename operator needs a caveat before these figures are read as change sensitivity. A rename mutant is excluded unless the baseline graph already holds a node for the path segment it renames, and it counts as detected when the renamed

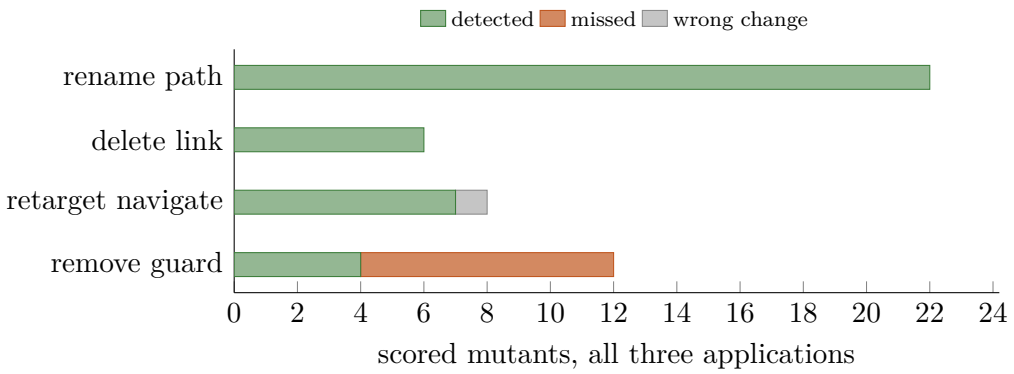


Figure 5.4: Mutation outcomes by operator, summed over realworld, alfresco and bitwarden. Excluded mutants are not shown. Every mutant that left the graph unchanged was a removed guard; the one wrong change was a retargeting.

segment appears in the graph. A screen is identified by its full path (Section 3.1), so a literal path the extractor has already parsed must yield a different node once renamed, and detection for this operator is close to guaranteed by construction. It supplies 22 of the 48 scored mutants and 22 of the 39 detections. What it tests is node extraction rather than the graph's response to a change in behaviour. The three operators that can fail are detected in 17 of their 26 scored mutants. On alfresco the attributed graph carries one edge, so its rate is a node-level result and not a statement about transitions.

Every mutant that produced no change in the graph at all was a guard removal: one in realworld, two in alfresco and five in bitwarden. Six of these eight are guards declared on a route whose path is the empty string. A second method confirms the defect directly. In realworld both children of the `editor` route declare `canActivate: [requireAuth]`, one under the path `':slug'` and one under the empty path. In the graph the edge entering `/editor/:slug` carries the guard `requireAuth` and the edge entering `/editor` carries no guard at all. The defect class *guards unreliable in both directions* of Section 5.2 therefore has a second confirmation specific to this case. The other two failures are guards whose enclosing route path the harness could not resolve, so they are reported as failures of the operator and are not attributed to this defect.

The eight failures divide by application as follows. One is in realworld, on the empty-path child of the `editor` route. Two are in alfresco, both on routes whose path is empty. Five are in bitwarden, of which three are on empty-path routes and two are the unresolved cases just described. The three applications do not declare their routes in the same way: realworld and alfresco export route arrays, while bitwarden declares its routes inside NgModule routing modules through `RouterModule.forRoot` and `forChild`. Empty-path guard misses occur under both styles.

The same mutants were also run against the baseline graph, which answers a question RQ3 leaves open: whether attribution changes how sensitive the graph is to source changes. For realworld the detection rate per scored mutant barely moves, from 92.3% to 95.2%, but the number of mutants that can be scored at all rises from 13 to 21, because attribution puts constructs into the graph that the baseline did not

contain. The clearest case is the link deletion operator: against the baseline graph all six of its candidates are excluded, since none of the links in the application shell produces a baseline edge, and against the attributed graph all six are scored and all six are detected. For alfresco and bitwarden the figures are the same on both graphs, 9 of 11 and 10 of 16, which matches the coverage results of Section 5.3, where alfresco gains nothing from attribution. Attribution therefore does not make the graph more sensitive per construct. It widens the part of the source the graph is sensitive to at all.

Of the 48 scored mutants, 39 were detected. The graph responds to changes in the constructs it represents, and the two places where it does not are the coverage gap, which the exclusions measure, and the representation of guards on routes with an empty path.

Chapter 6

Discussion

6.1 Key findings

Static extraction of a transition graph from Angular source fails in two different places, and which of the two dominates depends on the size of the application. On realworld and alfresco, the two smallest applications by construct count, node extraction recovers all or nearly all of the screens that the route configuration declares (Table 5.2). What is lost there is lost at the edge: the navigation sites exist in the source, the screens they would connect exist in the graph, and the extractor fails to connect them because it cannot decide which screens a site belongs to. On ghostfolio and mifos, the two largest applications by construct count, the loss starts one level earlier. A route whose path is a constant reference, and a route table behind a lazy boundary that the extractor does not cross, produce no node at all, and every navigation site that belongs to such a screen is lost together with every site that points at it. In two applications the baseline produces no transitions at all (Table 5.1): k8s dashboard yields dozens of nodes with no edges between them, and ghostfolio collapses to a graph in which none of its declared screens has a node (Table 5.2).

The two failures cap each other, and that is the structural result of the measurement. Attribution cannot draw an edge into a screen that has no node, so node extraction bounds what attribution can achieve. Node extraction cannot place a navigation site that lives outside the component its route renders, so attribution bounds what better node extraction would yield. Neither level can be repaired into a usable graph on its own. An evaluation that reported only one of the two would have described a tool that works better than it does, which is why Chapter 5 reports node coverage beside the conversion ratio instead of the ratio alone.

The ranking of the defect classes depends on which unit the corpus is counted in. Attribution is the largest class in most of the applications (Table 5.4), and node defects dominate the rest and also the corpus total, because the largest application contributes most of the constructs and loses them at the node level. Both views are correct and they answer different questions. The per-application view says what a developer would hit on a given codebase, and the corpus view says where the engineering effort pays off across codebases. Reporting only the total would have hidden the attribution problem that most applications show, and reporting only the per-application ranking would have hidden that one large application can dominate

everything that is aggregated.

The answer to RQ3 has to be read against what the loss rules can and cannot delimit. Because a lost construct is assigned to the first class that applies (Table 4.2), a construct inside a shared component that only appears in screens without nodes is counted as an attribution loss, although attribution alone cannot recover it. The measurement shows the converse as well: a majority of the deterministic gain came out of constructs the rules had assigned to node defects (Table 5.7). The attribution class is therefore not a bound in either direction, and the two levels of failure are not separable by the rules that classify them. On realworld, whose node extraction is complete, attribution recovered all of its attribution losses but one, and that one is in a service. The gap between the bound and what was recovered is accounted for by Table 5.8, and three of its reasons are not attribution problems at all: the target screen has no node, the owning component is never reached from any screen, or the site sits in a service or a guard that this version does not analyse. Precision on the deterministic tier did not fall, and no construct that the baseline converted was lost, so the coverage gain was not bought by widening the must tier. The must-tier violation is the same edge in both runs and predates attribution.

The part of the model that Chapter 3 specifies but does not implement has a measurable price. Navigation sites in services, guards, interceptors, resolvers and effects are counted separately, and in absolute terms they are a small class across the corpus, yet they are not spread evenly: in one application they are almost as large as the shared-component class, and in another they are the only attribution loss left after the implemented pass has run. The call graph with dependency injection is thus required for applications that navigate from services.

RQ4 gives a conditional answer, and the condition is RQ1. Renaming a route path was detected in every scored mutant in all three applications that were mutated. Deleting a `routerLink` was detected in every scored mutant, and retargeting a `Router.navigate` call in all but one of its own scored mutants. Every mutant that left the graph unchanged was a guard removal, and the one remaining miss is a retargeted `navigate` call in bitwarden whose old target the baseline had not resolved (Table 5.10). Most of the missed guards are declared on a route whose path is the empty string (Section 5.4). A second method confirms the mechanism independently of the mutation run: in realworld the edge entering `/editor` carries no guard although the route declares `canActivate`, while the edge entering `/editor/:slug` carries it. So a guard declared on an empty-path child route is not represented on the edges that enter that screen. This is a new instance of the class “guards unreliable in both directions” from Chapter 5, and that class now has two confirmations of its own, from two methods that do not share a mechanism.

Detection is conditional on coverage in a second sense, which the exclusion rule makes visible. Nearly a third of the sampled mutants could not be scored at all, either because the construct they change is absent from the graph or, for a guard, because no edge enters the screen it protects (Table 5.9). Those exclusions are not a sensitivity result; they are the coverage gap of RQ1 appearing in a different experiment. The defensible statement is therefore conditional: given that a construct is represented in the graph, a change to it is almost always visible in the graph, and

the probability that it is represented at all is what RQ1 measures. For change impact analysis this is the right shape of claim, because an impact query is only ever asked about the part of the application that the model represents, and the graph states which part that is through its witnesses.

Guard removals were missed in all three mutated applications, and Section 5.4 records the routing style of each of them: realworld and alfresco declare their routes as standalone arrays and bitwarden declares them in NgModule routing modules. The defect appears under both, so it is not a property of one routing style. That is the whole of the comparison the measurements support. The corpus was not stratified by routing style and the other three applications were not mutated, so nothing more general about the two styles follows from this thesis.

Two uses of the extracted graph are supported by what was measured. One is change impact analysis. Every edge carries the file and line of the construct that produced it, as an invariant of the model rather than as a convenience, so a set of changed lines can be mapped to the edges whose witnesses fall inside them and from there to the screens those edges leave and enter. This is the screen-level counterpart of what Chianti does with atomic changes, affected tests and methods [60], and it sits in the impact analysis tradition that Arnold and Bohner describe [4, 12, 43]. RQ4 is the evidence that the mapping reacts to real edits rather than only existing on paper: for the constructs the graph represents, an authored change in the source produces the predicted change in the graph.

The other is checking in continuous integration at no execution cost. The graph is recomputed from source on each commit, the extraction is deterministic and carries no state between runs (Section 4.3), and nothing has to be installed, served, authenticated against or crawled. Properties of the navigation structure can then be asserted on every commit in the same way a type check is: that a screen still has at least one inbound deterministic transition, that a screen that was reachable from the shell still is, that a route path a test depends on has not been renamed. A guard assertion is the one obvious member of that list the graph cannot yet carry, because RQ4 found guards on empty-path routes missing from the edges entering their screen, and an assertion is worth less than nothing if it passes for the wrong reason. A dynamically built model cannot be used this way at the same cost, because it needs a running application, test data and an exploration budget, and it has to be rebuilt for each version [50, 51, 36].

Both uses stop at the same place, which is runtime dynamic routing. Where a navigation target is computed from a value, where the route table is assembled at startup from configuration rather than declared, and where a user interface state is an overlay that no route describes, the static graph is either silent or over-approximates into the may tier, which carries no soundness claim. The graph is a lower bound on the navigation an application can perform, and it has to be consumed as one. A query that must not miss a transition cannot be answered from it alone, and the graph's own witnesses and declared approximations are what tell a consumer which queries those are.

6.2 Limitations and threats to validity

The results are measurements of one tool, on one corpus, with four methods, performed by the person who wrote the tool. Each of those four choices limits what the figures support.

Construct validity. The conversion ratio divides the constructs witnessed by a deterministic edge by all navigation constructs that the independent parser finds in non-test source. That denominator over-counts in two ways that both push the ratio down. It contains constructs that no transition graph could convert, in particular calls that only change the query string of the current screen, which Chapter 5 reports on a separate row rather than as a loss. It also contains constructs in code that the application never mounts, such as components whose selector no template uses, which the inspection of the remaining losses identified. Every reported conversion ratio is therefore a lower bound on the fraction of reachable navigation that the extractor recovers, and the true figure cannot be computed without deciding reachability for each construct by hand. The loss breakdown has a second construct threat. The rules assign each lost construct to the first class that matches, so the precedence order is part of the measure. A construct in a shared component that is used only inside screens with no node is an attribution loss under the present order and a node loss under the reverse order. The sizes of the classes are comparable within one application and under one order, which is what Chapter 5 claims, but they are not independent quantities, and a different order would move constructs between classes without any change to the tool.

Internal validity. Extraction runs on source trees whose dependencies are not installed, so the TypeScript compiler cannot resolve types that come from the framework or from third-party packages. Type-directed steps such as narrowing an injected receiver are weaker than they would be in a fully installed project, and the measured figures are properties of that configuration rather than of the analysis in principle. The mutation experiment has small samples per operator, in single digits for several of the operator and application pairs, with a seeded quota rather than exhaustive generation; one mutant moves a per-operator rate substantially, so the pattern of which operators are detected is more trustworthy than the rates themselves, and the published advice on reporting mutation experiments is followed for that reason [56]. The exclusion rule that removes mutants whose construct is absent from the graph is applied using the graph itself, which couples the sample to the tool's own coverage; the excluded mutants are reported rather than dropped silently, but the sample is not independent of the artefact under test. The end-to-end oracle extracts URLs from test source text, which under-samples click-driven navigation and can collect strings that no test ever visits, so it measures node existence and nothing finer. The independent route parser cannot follow the lazy specifiers of one application, which leaves that application's node coverage based on two screens and effectively uninformative. That application is also one of the two for which the extractor produces no edges at all, so the weakest oracle and one of the weakest extraction results fall together.

External validity. The corpus is six open-source Angular applications pinned

to single commits. Proprietary applications are not measured, and they may differ in size, routing style and the mechanisms they use for navigation, which is the threat that open-source corpora are known to carry [34, 35]. Two further limitations come from how the corpus was assembled. The selection is tool-driven: `SAP/spartacus` was rejected because its routing is configuration-driven in a way the extractor does not support (Section 4.1), so the corpus is biased towards applications whose routing the tool already handles, and a figure measured on it is likely to be optimistic with respect to the population of Angular applications. The candidate pool also showed an inverse relation between application size and end-to-end test density, so no application offers both a large route tree and execution-based ground truth. The consequence runs through the whole evaluation: the execution-grounded evidence comes from a medium-sized application, while the findings about scale come from applications that have no execution oracle at all. The results are Angular results. The graph model and the lattice do not mention Angular, and Section 6.4 argues about other routers, but that argument is not a measurement.

No oracle for the scope. Every method in Chapter 4 checks a node, an edge or a witness. None of them checks the screen set that attribution computes, which is the contribution of the thesis. A site attributed to too many screens produces edges that the witness check still passes, because the construct really is at that location and really does name that target, and the end-to-end oracle only visits URLs and so cannot see a source screen that should not have been there. The shell rule is capped at *may* where the mounting route is invisible (Section 3.3), which limits what an over-large scope can claim, but it is a design precaution and not a measurement. The scope is therefore the least directly validated part of the analysis, and the figures for it rest on the edges it produces agreeing with the other methods.

Oracle independence. The route parser shares no code with the extractor, which makes it independent and says nothing about whether it is right. Its first version did not look inside wrapper calls, did not follow a lazily loaded module to the routing module it imports, and walked lazily loaded tables without their prefix, and it therefore reported that every declared screen of mifos had a node; after the correction it finds a far larger set of declared screens of which under a tenth have a node, with the extractor unchanged in both runs (Section 4.3.2). The number that moved was the oracle's, not the tool's. A differential test also only shows that two implementations disagree, never which of them is right, and each disagreement in this study was resolved by reading the application source, by the same person who wrote both implementations.

Experimenter bias. I wrote the extractor, and I also designed the evaluation, selected the corpus and defined the defect classes. The mitigations actually applied are the following. The selection criteria were fixed before any results were obtained, and the rejected candidates are reported with their reasons. The construct counts and the declared screens come from a parser that shares no code with the extractor. The witness check is mechanical and re-parses the source at the claimed location. The mutation oracle is the authored delta, so it does not consult the tool's output to decide what the expected change is. The corpus commits, the extractor commit and the harness version are pinned, the whole evaluation re-runs with one command,

and the conversion, coverage, loss, remaining-loss, before-and-after and mutation tables of Chapter 5 are generated from the results file instead of being entered by hand. The defect-class table, the oracle table and the inspection of the unreached components are written by hand. Determinism and the absence of accumulated state were checked before any measurement. Seven protocol corrections are reported in full (Section 4.3.2), including the three that made the reported figures worse. What was not done is equally relevant: there was no blinding, no second annotator for the loss classification and no independent audit of the residual constructs that were inspected by hand, and the defect classes were defined after the first observations rather than in advance. The requirement of two independent confirmations per defect class is the strongest control that was applied to that last point, and it is the reason one class is still reported as a candidate.

6.3 Declared approximations

The thesis claims soundness only for the deterministic tier and only over the declared route set (Claim 3.2). Following the soundiness position of Livshits et al. [47], the claim is paired here with an explicit list of the constructs the analysis does not decide, so that a consumer of the graph can tell which queries it may ask. The list separates constructs that are in scope, where a failure is a defect that Chapter 5 counts, from constructs that are out of scope, where no completeness is claimed at all.

In scope are route entries whose path is a string literal together with the component they render, `routerLink` bindings whose value is a literal or an array of literals, calls to `Router.navigate` and `Router.navigateByUrl` with a literal target, the nesting relation obtained from element selectors resolved within the importing file's scope, the primary `<router-outlet>` with the shell rule that follows from it, and guards named on a route as labels on the edges entering that route's subtree. Everything in this group is the analysis's responsibility. The failures measured in Chapter 5 are failures inside it: route paths that are constant references are not resolved, route tables behind a lazy boundary are not followed, some nodes are not linked to the component that renders them, and a guard declared on an empty-path child route does not reach the edges entering its screen. These are bugs with known causes and they are listed in Section 6.5, not approximations to be declared away.

Out of scope are the following constructs, each with the reason it is approximated and the effect on the graph.

- **Components created programmatically, including dialogs and overlays.** An overlay is a user interface state that no route declares, so it has no node, and the component that implements it is reached from no template, so its navigation sites have the empty scope and contribute nothing. Both the state and everything inside it are absent from the graph.
- **Named outlets and targets of the form `{outlets: ...}`.** An auxiliary route adds a second active route and a second mount point, encoded in the URL [26]. The model of Chapter 3 represents this as a second mount, but the adapter does not implement it, and the end-to-end oracle shows the corresponding

URLs among its misses.

- **Navigation relative to the active route (`relativeTo`).** The target is a function of the route that is active when the call runs, so it has no single value in the source. This is the one item in this list that attribution makes approachable, since a scope is exactly the set of screens the site can fire from, and the target can be resolved once per member of that scope.
- **Templates passed as values (`TemplateRef`) and rendered elsewhere.** The nesting relation can no longer be read from element names, and recovering it becomes a dataflow problem over the values that carry the template. This is the same obstacle that Section 6.4 identifies for JSX.
- **Dispatch through NgRx effects and router event subscriptions.** The dispatch relation is specified in Chapter 3 and not implemented, so a navigation inside an effect is attributed to no screen. The cost is concentrated in the applications that use the pattern and is zero in the others.
- **Navigation inside signal effects and observable subscriptions.** The firing condition is a data dependence in a graph the runtime builds, not a control-flow condition in the program text (Sections 2.7.5 and 2.8). The model gives such edges the modality *unknown*, with the read set recorded as the symbolic guard; no pass in this version emits one.
- **Computed navigation targets.** A target built at run time from values cannot be enumerated statically in general. The analysis either emits no edge or over-approximates into the may tier, where a single unresolved call site can connect to every declared route, as one of the protocol corrections showed (Section 4.3.2).
- **Injection tokens with several bindings.** Where a token can resolve to more than one implementation, the analysis joins over all of them, and the resulting edges are capped at *may*. Resolution is relative to an injector, so the provider hierarchy is part of the input (Section 2.7.4).
- **Configuration-driven route registration.** Where the route table is assembled at startup from provider configuration instead of being declared as data, neither the extractor nor the independent parser recovers the routes. This approximation bounds the corpus as well as the tool, since it is why one candidate application was rejected (Section 4.1).
- **Attribute selectors in the nesting relation.** Only element selectors are matched, so a component instantiated through an attribute selector does not appear in the nesting graph.

Table 6.1 gives the prevalence of the approximated constructs in the three applications that were cloned for the mutation run. The in-scope kinds are not repeated, because the construct denominator of Table 5.1 already counts them with the parser. The counts come from `approx-counts.mjs`, which is part of the harness and runs over the application sources with tests excluded. It counts call sites, which is a different procedure from the parser that produces the construct denominator of Chapter 5, so the two are not comparable term by term. Three of the six applications were counted, so no corpus-level prevalence is claimed.

Two things follow from the counts. Set against the construct denominators of Table 5.1, which are 40, 34 and 132 for these three applications, the approximated

Table 6.1: Prevalence of the approximated constructs in three of the six applications, produced by `approx-counts.mjs` in the harness. The figures are call-site counts over the application sources with tests excluded, taken at the pinned corpus commits. They are a different unit from the construct denominator of Chapter 5, which comes from the route-oracle parser.

Construct	realworld	alfresco	bitwarden
`dialog.open`	0	3	0
auxiliary outlets	0	7	0
`relativeTo`	1	1	7
`TemplateRef`	2	0	11
`createEffect`	0	49	0

kinds are few in realworld and bitwarden, so the approximations do not explain the coverage figures of Chapter 5 for those two. In alfresco the approximated sites outnumber the 34 constructs, almost entirely through `createEffect`, so for that application the approximations cannot be set aside. The two numbers are not the same unit, since the denominator counts navigation constructs while the `createEffect` figure counts effect declarations, whether or not they navigate. The approximated kinds are also concentrated: auxiliary outlets appear in one application of the three, reactive effects only in that same application, templates passed as values mostly in another, and relative navigation everywhere but in small numbers. An approximation that costs nothing on one application can therefore be the dominant one on the next, which is an argument for declaring the list rather than reporting a single aggregate.

The call-site counts also understate the structural cost of an approximation. A handful of `dialog.open` call sites can open many different components. The inspection of the owner components that attribution never reaches found about sixty components across the corpus that are opened programmatically, most as dialogs, whereas the grep finds three `dialog.open` call sites, all in alfresco, in the three applications it covered. The two figures cover different scopes and are not directly comparable. The right unit for the cost of this particular approximation is the number of components and navigation sites it strands, not the number of calls that strand them.

6.4 Transfer to other frameworks

All measurements in this thesis concern Angular. How much of the approach would carry over to React with React Router and to Vue with Vue Router [59, 78] is argued below without measurement.

What follows is a qualitative theoretical analysis and not a measurement. It compares the documented behaviour of the three routers with the input relations that the analysis needs, and what it yields are hypotheses about where the work would move, at the level of evidence of a design argument. Nothing in it shows that

Table 6.2: Constructs that provide the input relations, by router.

Concept	Angular	React Router	Vue Router
Outlet	`<router-outlet>`	`<Outlet />`	`<router-view>`
Several outlets	named outlets, encoded in the URL	none	named views, not in the URL
Declarative link	`routerLink`	`<Link to>`	`<router-link :to>`
Imperative navigation	`Router.navigate`	`useNavigate()`; redirect from a loader	`router.push`
Route guard	per route (`canActivate`, ...)	loader that throws a redirect	per route (`beforeEnter`) and global (`beforeEach`)
Access to services	hierarchical injectors	context and hooks	`provide`/`inject`, composables, plain imports
Reactive trigger	signal effect	`useEffect`	`watch`, `watchEffect`

the approach performs on React or Vue applications as it does on Angular ones. Establishing that would require a corpus for each framework and the methods of Chapter 4.

The analysis has a framework-independent part and a framework-specific part. The graph model, the lattice of scopes, the fixpoint, anchored edges and the expansion query do not mention Angular. What is specific to Angular are the four input relations of Section 3.2: component nesting, router outlets, the call graph with dependency injection, and dispatch relations. Moving to another framework means deriving the same four relations from that framework's constructs. Table 6.2 lists the corresponding constructs.

Outlets transfer directly: in all three routers a parent route renders a placeholder where the matched child is inserted. The components above the placeholder form a shell that is present on every child screen, so the scopes $\top$ and $\mathit{subtree}(r)$ have the same meaning. Vue's named views are simpler than Angular's named outlets, because they do not appear in the URL; they add a second mount point but no auxiliary route. React Router has no equivalent, so this class of defect does not arise there.

The call graph is easier in React and Vue. The hardest Angular-specific step is resolving an injected service through the injector hierarchy. In React, a navigation function is usually obtained from `useNavigate()`, and a hook can only be called while a component renders, either directly or through other hooks. Every such navigation site therefore has an owning component that can be found by following hook calls, and no injector needs to be resolved. Vue composables behave the same way inside `setup`. The case that remains is code that imports the router instance directly, which Vue applications often do in stores and HTTP interceptors. This

is the analogue of Angular's service case: the site has no component of its own, and its scope is the join over its callers. Since the link is a plain module import, it can be resolved with ordinary import resolution, as in the JavaScript call graphs of Feldthaus et al. [23].

Guards move to different scopes. A redirect thrown from a React Router loader fires when its route, or any route below it, is matched. It plays the role of `canActivate` and is anchored at $subtree(r)$. Vue's global `beforeEach` guard has no direct Angular counterpart. A redirect from it is anchored at $\top$, which the lattice already contains.

Component nesting is harder in React. Angular and Vue keep templates in a separate language, so the nesting relation can be read from element names. In React, components are created in JSX, which is ordinary JavaScript. A component can be stored in a variable, passed as a property or as `children`, and rendered under any condition the language can express. Recovering the nesting relation then becomes a data flow problem in its own right, closer to points-to analysis than to reading a template. This is the main place where a React version of the analysis would need new work.

The reactive boundary is the same. A navigation inside `useEffect` or a Vue watcher fires when its dependencies change, so, like a signal effect in Angular, it would be given modality *unknown*.

The lattice, the fixpoint and the anchored representation transfer unchanged. Of the input relations, outlets and guards transfer with small changes, the call graph becomes simpler, and component nesting becomes harder in React. These are expectations derived from the routers' documented behaviour. Whether they hold on real applications would need a corpus and the same evaluation, which is left for future work.

6.5 Future work

An oracle for the screen set is the cheapest addition that would strengthen the thesis most, because the scope is the contribution and Section 6.2 records that no method validates it directly. On an application small enough to read, the screens that render each navigation site can be enumerated by hand and compared with the computed scope, which would give attribution the kind of ground truth the end-to-end oracle gives node extraction. Recording, for each attributed site, the relation that supplied its scope would make the same comparison possible per relation.

The two defects that cap the larger applications are the most urgent, because nothing else in the analysis can be evaluated properly while they stand. Resolving constant-valued route paths means following a path expression to its definition and folding it, which is a local analysis over the route tables and would turn the application that currently collapses to a single node into one with a full node set. Following lazy boundaries means resolving a `loadChildren` or `loadComponent` specifier to the file it names and continuing route extraction there, including through a routing module that a lazily loaded module imports. The independent route parser follows lazy

boundaries after its correction (Section 4.3.2), and it finds 84 declared screens in ghostfolio, where every route path is a constant reference (Table 5.2), so the work is known to be feasible on this corpus, and the parser's output gives the expected result to check against. The gain is not limited to the nodes recovered. A large part of the attribution losses that remain after attribution are constructs whose scope was found but whose target screen has no node, and those are repaired by node extraction rather than by any change to the attribution pass.

The part of the model that Chapter 3 specifies and the prototype leaves out comes next: the call graph with dependency injection, and the dispatch relations that carry NgRx effects and router event subscriptions. The remaining losses in services and guards quantify what the omission costs on this corpus, which makes the work measurable rather than speculative. Resolution would follow class hierarchy analysis against Angular's provider declarations, in the style the framework-modelling literature recommends [17, 65, 3], and the fixpoint and the complexity bound for it are already stated. Guards that return a `UrlTree` belong to the same group, since a redirecting guard is itself a navigation site and not only a label.

RQ4 exposed one defect that is cheap to fix and worth fixing before the graph is used for anything: a guard declared on a route whose path is the empty string does not reach the edges entering that screen. The repair is in how the guard set of a screen is assembled from the route chain, and the test for it is already written in the form of the mutation operator that removes a guard. The wider treatment of guards in Chapter 3, where the guard of an edge is the conjunction of the conditions on the containment path with the guards of the target route, is the more substantial piece of work and would subsume it.

Dialogs and overlays should become user interface states. They are the largest declared approximation in structural terms, they strand entire components rather than single call sites, and the model needs only a second kind of node together with edges that open and dismiss it. The route-derived definition of the screen set would have to be widened, which is the reason this is not a small change to the adapter.

The runtime tier is the complement to the whole static approach rather than an extension of it. The model already admits observation identifiers as witnesses alongside source locations, so an observed transition can enter the graph without being confused with a derived one. Setting that tier against the dynamic lineage, where the state flow graph comes from firing events on a running application [50, 51, 72], would answer the question this thesis cannot: how much of what a crawler finds the static graph already contains, and how much of what the static graph contains a crawler never reaches. The Android work has the same structure, falling back on runtime traces exactly where static attribution of shared code fails [77].

The corpus needs to grow in three directions. An automated search pipeline over a larger pool of repositories, with the published filters for repository quality applied mechanically [34, 35], would replace a hand-screened set of ten candidates and would weaken the tool-driven selection bias of Section 6.2, provided that applications the extractor handles badly are kept in the corpus rather than rejected. A case study on a proprietary industrial application, reporting only aggregate figures, is the most direct answer to the external validity threat, since enterprise routing style is exactly

what an open-source corpus cannot show. A minimum density of user interface tests should become an explicit selection criterion whenever execution-based ground truth is needed, since the candidate pool showed that this density falls as applications grow, and without such a criterion a study ends up with its oracle and its scale on different applications.

A React or Vue adapter, evaluated with the same harness on a corpus of the same shape, would turn Section 6.4 from an argument into a result. The predictions are specific enough to be falsified: outlets and guards should transfer with small changes, the call graph should become easier because a navigation function is obtained through a hook or a composable, and component nesting should become harder in React because JSX makes the nesting relation a dataflow problem.

The defect class "node not linked to its component" still has one confirmation, from the comparison between the route parser and the graph, and the rule adopted in Chapter 5 admits a class only with two. A second method has to reach it independently, and the end-to-end oracle is the natural candidate, since a screen whose node exists but carries no component link should still be visited by tests while contributing no outbound edges. Until then the class stays a candidate, and the share of the corpus total that it accounts for stays provisional with it.

The use that motivated the thesis is still unmeasured. Agents that drive a user interface or a repository work without a navigation model: mobile testing agents pass the current screen to a model and replay the action it returns [46, 70], application-specific knowledge for them is obtained by dynamic exploration [71, 69], and coding agents navigate a repository through an interface whose design measurably changes their performance [31, 74, 79]. Whether a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge, improves either kind of agent, or improves automated test generation, is a controlled experiment that this thesis does not attempt. The graph and the harness are the artefacts such an experiment would need, and the defects quantified in Chapter 5 are what it would have to control for, since a model that is missing half of an application's screens would confound the comparison.

Chapter 7

Conclusion

This thesis set out to measure how much of the navigation structure of an Angular application can be recovered from its source code without running the application, to explain where that recovery fails, and to repair the largest class of failure with an analysis that attributes a navigation site to the screens from which it can fire. The selection criteria for the corpus were fixed before any results were obtained. The protocol was corrected seven times while it was being developed, including the definition of the conversion ratio and of its denominator (Section 4.3.2), and the result tables of Chapter 5 come from the harness run on pinned commits.

The answer to RQ1 is that the baseline extractor witnesses between 0.0% and 15.0% of the 1820 navigation constructs of the corpus with a deterministic transition, and that the loss occurs at two levels, because some declared screens never become nodes at all and most navigation sites in the screens that do become nodes are never turned into edges. The answer to RQ2 is that the failures fall into five defect classes, each confirmed by at least two methods that do not share a mechanism, together with one candidate class that has a single confirmation, and that assigning every lost construct to exactly one class makes attribution the largest class in four of the six applications while node defects, unresolved constant route paths and unfollowed lazy boundaries dominate the other two and the corpus total. The answer to RQ3 is that scope attribution raises the conversion ratio in four of the six applications, from 15.0% to 37.5% on realworld and from 5.4% to 10.0% on mifos, converts 81 of the 468 constructs that the loss rules assign to it, 38 of them deterministically, together with a further 41 deterministic conversions of constructs the rules had assigned to node defects, and costs nothing in deterministic precision, since the one must edge that fails the witness check after attribution is the same edge that failed before it. The answer to RQ4 is that the graph is sensitive to authored source changes in the constructs it represents, with 39 of 48 scored mutants detected, a route path rename detected in every scored mutant of all three mutated applications, a deleted link detected in every scored mutant, and a retargeted `navigate` call detected in seven of eight. Eight of the nine misses are a guard removal, six of those eight on routes whose path is the empty string, and the ninth is a retargeted `navigate` call whose old target the baseline had not resolved.

The thesis contributes five things. Attribution is formulated as a dataflow analysis over the powerset lattice of screen sets, computed as a fixpoint over four input relations, with a soundness property for the deterministic tier that is stated

and mechanically tested, though not proved, over the declared route set and is paired with an explicit list of the constructs the analysis does not decide (Claim 3.2, Section 6.3). The analysis is implemented in *uigraph*, an open-source extractor with a framework-independent core and an Angular adapter that records the rule and the source location that produced each node and edge. Static navigation extraction is measured on a corpus of six Angular applications pinned to fixed commits, with four methods chosen so that they do not share a blind spot: a witness check that re-parses the source at each claimed location, a differential test against an independently written route parser, an end-to-end oracle taken from the applications' own passing tests, and a mutation experiment whose ground truth is the authored change. The failures are classified into defect classes that are admitted only on two independent confirmations, and every construct the extractor loses is assigned to exactly one class, which makes the classes comparable within an application and across the corpus. The corpus and the harness are reusable artefacts: the evaluation re-runs with one command against pinned commits of the extractor and of every application, and the conversion, coverage, loss, remaining-loss, before-and-after and mutation tables of Chapter 5 are generated from the results file. The defect-class table, the oracle table and the inspection of the unreached components are written by hand.

What the measurements establish is that a declared route table is a usable starting point for static navigation extraction and not a sufficient one. The two levels of loss bound each other, so neither node extraction nor attribution can be improved into a usable graph on its own, and scope attribution recovers nearly all of the attribution losses on realworld, where node extraction is complete, and far fewer on the larger applications, where node defects limit it. Where a construct is in the graph, a change to it is almost always visible in the graph, which is the property that change impact analysis and commit-time checking need; where it is not, no change to it is visible, and the coverage gap RQ1 measures is what the excluded mutants of RQ4 report a second time.

What they leave open is also specific. Constant-path resolution and lazy-boundary following are defects with known repairs that the independent parser already performs, so what the extractor would recover with them in place is unmeasured. The call graph with dependency injection and the dispatch relations are specified and not implemented, and the remaining losses in services and guards state what the omission costs rather than what closing it would yield. Guards on empty-path routes are represented wrongly, and one defect class still waits for its second confirmation. Whether a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge, helps the consumers that motivated the work, agents that drive a user interface or a repository and automated test generation, is not measured here; the extractor, the corpus and the harness are what such a measurement would start from, and the defects quantified in Chapter 5 are what it would have to control for.

Bibliography

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[27] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[28] May Haydar, Alexandre Petrenko, and Houari Sahraoui. Formal verification of web applications modeled by communicating automata. In *Formal Techniques for Networked and Distributed Systems (FORTE)*, volume 3235 of *Lecture Notes in Computer Science*, pages 115–132, 2004.

[29] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[30] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[32] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[33] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[36] Iva Kertusha, Gebremariem Assress, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry, 2025. arXiv:2503.05378.

[37] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[38] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[39] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[40] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[41] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[42] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[43] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[44] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[45] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[46] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[47] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[48] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[49] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[50] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[51] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[52] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[54] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[55] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[56] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[57] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[58] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[59] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[60] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[61] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[62] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[63] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[64] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[65] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[66] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[67] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[68] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. In *Proceedings of the ACM on Software Engineering (FSE)*, 2025.

[69] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[70] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[71] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[72] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 2022. IEEE Xplore document 9765776.

[73] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[74] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces

enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[75] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[76] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[77] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[78] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[79] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, Singapore, 2023. Association for Computational Linguistics.

[80] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.